

NOTTINGHAM TRENT UNIVERSITY
SCHOOL OF SCIENCE AND TECHNOLOGY

**Evaluating Flow State for Players with Information
Processing Difficulties: A Fuzzy DDA Algorithm**

by

Samuel Gillary

in

2026

**Project report in part fulfilment
of the requirements for the degree of
Bachelor of Science with Honours**

In

Computer Science (Games Technology)

I hereby declare that I am the sole author of this report. I authorize the Nottingham Trent University to lend this report to other institutions or individuals for the purpose of scholarly research.

I also authorize the Nottingham Trent University to reproduce this report by photocopying or by other means, in total or in part, at the request of other institutions or individuals for the purpose of scholarly research.

Signature

A handwritten signature in black ink, reading "S. Gillary". The signature is written in a cursive style with a large, stylized 'S' and a long, sweeping underline that extends under the word "Gillary".

Samuel Gillary

ABSTRACT

Contemporary video games depend on a delicate equilibrium between player capability and game requirements to sustain the state of cognitive engagement known as 'Flow'. For individuals exhibiting slower processing speeds or reaction times, stringent timing constraints may compromise this equilibrium and impose unnecessary barriers to accessibility. This study investigates whether an automatic, performance-driven Dynamic Difficulty Adjustment (DDA) system can alleviate these obstacles and facilitate sustained Flow. A third-person action prototype was developed within the Unity platform, employing an Accord.Fuzzy controller to modify enemy behavior contingent upon real-time reaction performance. To evaluate the system objectively, EEG data were procured via an OpenBCI Galea headset to compute an Engagement Index (EI) across three participants. The findings indicate that the adaptive system effectively decreased cognitive load for a neurodivergent participant; however, it was insufficiently challenging for a neurotypical participant and underscored the susceptibility of EEG signals to movement artifacts. The research concludes that although performance-based fuzzy DDA functions effectively in the background, future accessible game design should incorporate hybrid systems that combine behavioral and biometric measures to more precisely respond to a player's internal cognitive state.

ACKNOWLEDGEMENTS

Peter Blanchfield – Project Supervisor

Sean Haddick – assisted with the consent forms, and supervised the Galea recording session

Dan Buxton – performed data analysis and supervised the Galea recording session

Adam Wallbanks – supervised the Galea recording session

All participants of the study

Jan Yelland-Sutcliffe – DSA-provided Study Skills Mentor

Peter Ashby – DSA-provided Specialist Mentor

CONTENTS

ABSTRACT.....	II
ACKNOWLEDGEMENTS	III
LIST OF FIGURES	VIII
CHAPTER 1.....	1
INTRODUCTION	1
1.1 Introduction	1
1.2 Aims	2
1.3 Project Plan and Report Structure	2
CHAPTER 2.....	4
CONTEXT	4
2.1 Introduction	4
2.2 The Flow State	5
2.3 Dynamic Difficulty Adjustment algorithm	10
2.4 Relevance.....	13
2.4.1 Case Study: <i>Resident Evil 4</i> (2005)	13
2.4.2 Case Study: <i>Celeste</i>	17
CHAPTER 3.....	20
NEW IDEAS	20
3.1 Key Research Focus.....	20

3.2	Prototype Design and Functional Requirements	21
3.2.1	Prototype Overview	21
3.2.2	Prototype Design	21
3.2.3	Fuzzy Logic Controller	25
3.2.4	Requirements Analysis	26
3.3	Methodology.....	28
3.3.1	Apparatus and Instrumentation	28
3.3.2	Experimental Procedure	29
3.3.3	Data Analysis and The Engagement Index.....	29
3.3.4	Mitigation of Extraneous Variables	31
3.4	Achieving Project Aims.....	32
3.5	Methodology Critique	33
CHAPTER 4.....	34	
INVESTIGATION	34	
4.1	Project Planning and Methodology	34
4.1.1	Development Methodology	34
4.1.1	Timeline and Workload Management	35
4.1.2	Risk Management and Contingencies.....	36
4.2	System Architecture and Software Design.....	36
4.2.1	High-Level Technologies	36

4.2.2 Object-Oriented Design (OOD)	37
4.2.3 HCI and Usability Design	37
4.3 Prototype Development.....	38
4.3.1 Core Mechanics and Environment.....	38
4.3.2 Enemy AI and State Machine.....	39
4.3.3 Implementing the Fuzzy Logic Controller	41
4.4 Software Testing and Verification	44
4.4.1 Testing against requirements	44
4.4.2 System Testing.....	45
4.5 Development Process Evaluation.....	46
4.6 Carrying out the investigation	47
4.6.1 Setup	47
4.6.2 Execution.....	48
4.6.3 Data Analysis	49
4.6.4 Evaluation.....	50
CHAPTER 5.....	52
RESULTS / DISCUSSION	52
5.1 Introduction	52
5.2 Research Results and Statistical Analysis.....	52
5.3 Evaluation of Process	56

5.3.1	Successes and Opportunities	56
5.3.2	Limitations and Challenges	57
5.4	Evaluation of the Prototype	57
CHAPTER 6.....	59	
CONCLUSIONS / FUTURE WORK	59	
6.1	Conclusions	59
6.2	Future work.....	60
6.3	Legal, Social, Ethical and Professional Issues	62
6.4	Synoptic Reflections	64
REFERENCES	66	
APPENDIX A	I	
APPENDIX B.....	III	
APPENDIX C.....	IV	
APPENDIX D	V	
APPENDIX E	VI	
APPENDIX F	VII	
APPENDIX G	VIII	
APPENDIX H	IX	
APPENDIX I	X	
APPENDIX J	XII	

LIST OF FIGURES

Figure 1	7
Figure 2.	16
Figure 3.	17
Figure 4.	22
Figure 5.	23
Figure 6.	24
Figure 7.	25
Figure 8.	35

CHAPTER 1

INTRODUCTION

1.1 Introduction

Human-computer interaction in gaming fundamentally depends on maintaining a delicate equilibrium between player capabilities and game demands. When this balance is achieved, players can attain a highly focused and rewarding state known as "Flow," as articulated by Csikszentmihalyi (2008). Nevertheless, for players with neurodivergent profiles or conditions that affect processing speed and reaction times, stringent timing constraints frequently serve as unnecessary obstacles. These instances demanding precise timing can interrupt the flow, resulting in stress or frustration—not due to deficiencies in skill, but because the pacing of the game fails to synchronize with their cognitive rhythms.

Developers have traditionally employed static accessibility options, such as slowing gameplay or enlarging input windows. While these methods are beneficial, they rely on players identifying the issue, pausing the game, and manually modifying settings, which can disrupt immersion. Dynamic Difficulty Adjustment (DDA) offers a more seamless alternative by automatically modulating the level of challenge. However, most commercial DDA systems rely on fundamental performance metrics like health, accuracy, or survival time, which indicate the player's actions but do not necessarily reflect their perception of the experience.

This project explores whether a flexible, multi-measure strategy can more effectively support players who encounter difficulties with strict timing. A bespoke third-person action prototype was developed utilizing a Fuzzy Logic

controller to dynamically adjust difficulty levels in real-time, based on psychomotor performance. To evaluate whether these modifications genuinely facilitated the player's cognitive state, EEG hardware was employed to monitor brainwave activity and calculate a continuous Engagement Index (EI). By contrasting the system's automatic difficulty adjustments with objective neurophysiological data, this research evaluates the efficacy of performance-based Dynamic Difficulty Adjustment (DDA) in maintaining optimal flow without necessitating manual intervention in the user interface.

1.2 Aims

Aim 1: Develop an autonomous Dynamic Difficulty Adjustment (DDA) system capable of interpreting player performance without explicit UI interaction.

Aim 2: Objectively validate the efficacy of the DDA system in sustaining a player's Flow State using neurophysiological biometric tracking.

1.3 Project Plan and Report Structure

Considering the limitations imposed by the academic calendar, the project transitioned from an incremental approach to a more expedited Agile-Scrum methodology. This adjustment enabled the development process to accommodate overlapping deadlines and the inherent unpredictability associated with constructing a comprehensive investigation, including a prototype. The organization of the report mirrors this evolution, systematically guiding the reader through the conceptual, technical, and evaluative phases of the project.

- **Chapter 2 – Context:** Establishes the theoretical foundation, exploring Flow State, psychomotor accessibility, and the shortcomings of existing Difficulty Adjustment systems in both research and commercial games.

- **Chapter 3 – New Ideas:** Introduces the project’s methodological approach, outlining the design of the fuzzy-logic controller and the rationale for integrating EEG-based validation into the adaptive model.
- **Chapter 4 – Investigation:** Brings together practical engineering and the experimental study. This chapter covers the construction of the Unity prototype, the integration of the Accord.NET fuzzy-logic system, the setup of the Finite State Machine, and the full investigation procedure, including participant recruitment, EEG data collection, experimental setup, and the challenges encountered during live testing.
- **Chapter 5 – Results and Discussion:** Presents the Engagement Index data alongside participant feedback, analysing how effectively the prototype maintained flow and where the adaptive system struggled to respond to individual differences.
- **Chapter 6 – Conclusions and Future Work:** Summarises the project’s key findings, addresses the Legal, Social, Ethical, and Professional Issues (LSEPIs) associated with biometric research, and offers synoptic reflections on the technical and personal development gained throughout the project.

CHAPTER 2

CONTEXT

2.1 Introduction

Modern video games have evolved far beyond simple interactive entertainment; they are complex, highly demanding environments that require continuous cognitive and psychomotor engagement. The core of successful game design rests on a delicate balance between the challenge presented to the player and the player's physical or mental skill. When this balance is maintained, games have the unique ability to capture a player's absolute focus. However, when this balance fails, whether due to rigid difficulty structures, accessibility barriers, or poor pacing, the player's experience can quickly deteriorate into frustration or boredom.

To understand why advanced Dynamic Difficulty Adjustment (DDA) is necessary, it is essential to first examine the psychological phenomena that govern player engagement. This chapter explores the foundational theories of play, the fragility of optimal psychological states, and how contemporary developers have historically sought to mitigate accessibility barriers through static toolkits and performance algorithms. By analysing these existing frameworks, a clear critical gap emerges in how systems monitor and respond to a player's true internal state.

2.2 The Flow State

Video games are uniquely positioned as modern engines for psychological engagement. Unlike passive media, they demand active participation within a strictly defined boundary. As Huizinga (1951) denotes in *Homo Ludens*, there are some characteristics of play. The first is that "it is free, is in fact freedom" (p. 8). The second is that play "is not 'ordinary' or 'real' life" (p.8). Nevertheless, more relevantly:

"[T]he third main characteristic of play: its secludedness, its limitedness. It is 'played out' within certain limits of time and place. It contains its own course and meaning" (Huizinga, 1951, p. 9).

This conceptual seclusion is a fundamental prerequisite for play. In computer games, this boundary is often negotiated physically via the controller or mouse and keyboard, and visually through the screen, creating a protected space that suspends the unpredictable realities of the external world. Huizinga (1951) categorises such isolated, rule-bound "play-grounds", listing arenas, temples, and the "magic circle", as spaces where special rules obtain. While Huizinga was writing as a cultural historian, it was not until decades later that this observation was formalised for contemporary media. Formally adapted as a game design concept by Salen and Zimmerman (2003), the "magic circle" serves as the definitive architectural boundary in modern game studies, defining the boundary between the real world and the artificial locus of play (p. 119).

Establishing this architectural boundary is critical because it dictates the psychological capacity of the player. Once the magic circle successfully insulates

the user from external anxieties and distractions, it fosters an environment for optimal cognitive performance. It is within this protected locus of play that a player can achieve what Csikszentmihalyi (2008) defines as "Flow". Originally published in 1990, in his seminal work, *Flow: The Psychology of Optimal Experience*, Csikszentmihalyi defines the concept as:

"The state in which people are so involved in an activity nothing else seems to matter; the experience itself is so enjoyable that people will do it even at a great cost, for the sheer sake of doing it." (Csikszentmihalyi, 2008, p.4)

This intrinsic motivation, where an activity is pursued strictly for its own reward, is what Csikszentmihalyi characterises as an "autotelic" experience (Csikszentmihalyi, 2008, p. 67). Deriving from two Greek words, *auto* (self) and *telos* (goal), an autotelic activity is entirely self-contained, of which no expectation of reward or future benefit is present. This may explain why players voluntarily subject themselves to high-performance gaming, experiencing intense, frustrating, or mechanically exhausting challenges. Once the magic circle has been established, the reward is no longer derived from external validation but from the act of play itself. In order for a game to successfully transform into an autotelic experience, motivation alone is insufficient. As Csikszentmihalyi outlines, the game must provide a strict structural framework to sustain the player's attention, relying primarily on clear goals, immediate feedback, and a delicate balance between the challenge presented and the player's mechanical skill.

Of these conditions, the balance between challenge and skill is the most critical for maintaining high-performance engagement. Csikszentmihalyi (2008) visualises this variable equilibrium in his Flow Channel Diagram (see Figure 1). According to the model, flow exists within a narrow psychological corridor. If a

game's mechanical challenge significantly exceeds the player's current skill level, the player is pushed out of this optimal state and into a state of anxiety. Contrarily, if the player's skill outpaces the challenge presented by the game, then they may slip into a state of boredom. Thus, to sustain an autotelic experience, a video game cannot rely on static difficulty. It must operate as a responsive system that continuously scales its mechanical friction to match the player's increasing proficiency, keeping them firmly within this active channel.

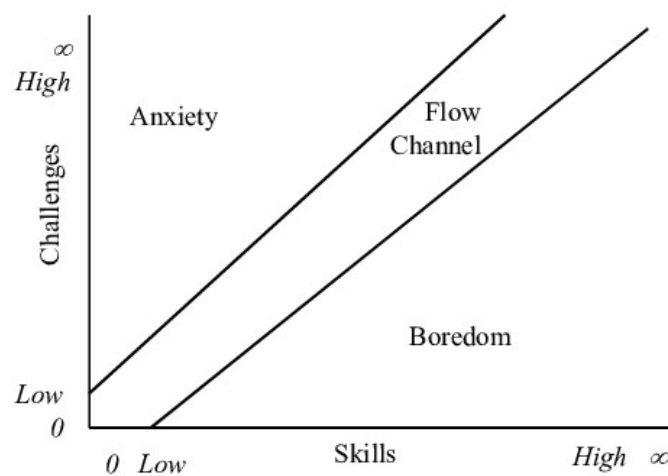


Figure 1. Flow Channel Diagram. Source: Dickson and Dolnicar (2004).

While Csikszentmihalyi's model assumes a normative baseline of physical capability, applying this framework to psychomotor accessibility unveils a critical rupture in the architecture of modern play. With the controller being the physical threshold into the 'magic circle'; it is also the tool in which player intention is translated into digital action. Even so, when a game requires fast, time-sensitive inputs, this controller may become a temporal bottleneck for players with delayed processing speeds. This temporal friction artificially inflates the "Challenge" axis of the Flow Channel. Regardless of whether a player

understands the required action, the inability to actuate the controller within a strict timing window creates a severe disconnect between intention and execution. Consequently, the player is perpetually forced into the “anxiety” zone (Csikszentmihalyi, 2008, p.74), punished by the game’s arbitrary pacing rather than tested on their strategic understanding. Therefore, accessibility tools (such as settings to modify game speed or widen input windows) are not merely “cheats” or difficulty modifiers; they are structural interventions required to realign the challenge-skill balance, finally allowing players the opportunity to sustain an autotelic experience.

These accessibility-related distortions demonstrate just how fragile the flow state can be, and how easily a player may be pushed out of the optimal channel without any change in their underlying skill. To study these shifts accordingly, researchers must therefore turn to reliable methods of identifying when someone is in flow. This has led to a sustained focus on how flow can be measured, and what indicators best capture its presence during gameplay.

Recognising how easily how players can be pushed out the flow channel makes the issue of measurement unavoidable, leading to a key question for researchers: how can flow be captured and quantified?

Csikszentmihalyi (1975) lists six elements of the flow experience:

1. Merging of action and awareness: Csikszentmihalyi describes this as “perhaps the clearest sign of flow”. The person in flow has no awareness of themselves as separate from the action they are performing (p. 38).
2. Centring of attention: The ability to completely focus on a task with narrow focus and shut out any distracting stimuli from the outside world (p.40).

3. Loss-of-ego/self-forgetfulness: The person does not need to use the self to complete the activity; where the demands of the task outweigh the person's "self-ish" considerations (p.42).
4. Control over actions and the environment: The person has no active awareness of control, albeit undisturbed by the loss thereof (p.44).
5. Coherent demands and unambiguous feedback: Coherent, noncontradictory demands and clear feedback created ordered rules. Consequently, action and evaluation become automatic; the person is too involved with experience to stop and reflect (p.46).
6. Its "autotelic" nature: As defined earlier, the flow experience needs no goals or rewards, as the task is rewarding in itself (p.47).

Since these components describe a highly internalised, subjective psychological state, objectively quantifying flow during human-computer interaction (HCI) and gaming has historically presented a significant methodological challenge.

Traditionally, researchers have relied on qualitative self-reporting data to measure the experiences of the participants. For example, the Flow Short Scale (FSS), originally developed by Rheinberg, Vollmeyer and Engeser (2003), is highly effective at capturing a participant's perceived experience and providing some form of measurable score for the potential of an activity to cause flow. However, relying exclusively on subjective self-reporting introduces inherent limitations. Because post-session surveys capture only a summary of a finished experience, Nacke and Lindley (2008) argue that states like flow remain "informal terms" until they are grounded in continuous, "measurable criteria" that can track the dynamic fluctuations of active gameplay (p.81).

To circumvent these limitations, contemporary research has increasingly shifted toward real-time physiological and neurophysiological tracking, allowing

researchers to observe the biological markers of flow without interrupting the participant or relying solely on retrospective qualitative data.

One such method of physiological monitoring is through the utilisation of electroencephalography (EEG). This refers to the “registration of neuronal activity within the human brain” (Kirschstein and Köhling, 2009). EEG signals arise from the summed electrical activity of large populations of cortical pyramidal neurons, whose synaptic inputs create voltage fluctuations detectable at the scalp (Kirschstein and Köhling, 2009). These electrodes are integrated into devices like the OpenBCI Galea headset (OpenBCI, 2024).

2.3 Dynamic Difficulty Adjustment algorithm

To achieve the necessary responsiveness required to maintain the ‘magic circle’, developers have increasingly tuned to Dynamic Difficulty Adjustment (DDA). As formally defined by Zohaib (2018), DDA is:

“...a method of automatically modifying a game’s features, behaviours, and scenarios in real-time, depending on a player’s skill, so that the player, when the game is very simple, does not feel bored or frustrated, when it is very difficult.”

By adhering to this principle, DDA offers a reactive alternative to traditional static difficulty settings (often presented as arbitrary ‘Easy, Normal or Hard’ settings) that force players to pre-determine their skill level. Formalised by Hunicke (2005), these adaptive systems actively monitor player performance in real-time to execute invisible mechanical scaling, mitigating those exact extremes of frustration and boredom. Hunicke (2005) argues that abstracting the difficulty curve away from the player’s conscious control preserves the integrity of the game’s pacing, keeping the player immersed in the challenge rather than the user interface.

Building on Hunicke's call for systems that dynamically preserve the challenge-skill balance, subsequent research has turned toward identifying the concrete signals that can drive such adaptation. Seyderhelm and Blackmore (2021) offer a comprehensive review of these signals, outlining the various measures used to trigger difficulty adjustments in modern games. Their analysis focuses specifically on the use of DDA within serious games.

Serious games often aim to teach, train, or rehabilitate, making appropriate calibration of challenge essential. Seyderhelm and Blackmore argue that when games are designed with a learning or behavioural outcome in mind, player engagement is not merely a metric of enjoyment but a prerequisite for effective skill acquisition. If the difficulty is too high, learners get agitated and disconnect; if it is too low, the task loses its educational value and no longer supports progression. For this reason, the authors position DDA as a pedagogical tool rather than a feature intended solely for entertainment. They emphasise that adaptive systems should continuously monitor player performance and cognitive state to keep them within the range of productive challenge.

Seyderhelm and Blackmore (2021) state that there are two elements to each system. The first is "what is measured to trigger the DDA mechanism" (p.12), and the second is "how the adaption mechanism is implemented" (p.12). Based on these dimensions they categorise real-time adaptations into three broad types:

- Performance adaptation: The game alters itself based on the performance of the player.
- Physiological adaptation: Physiological systems took metrics and fed them into the DDA to trigger difficulty adjustment.
- Cognitive adaptation: Systems that measure cognitive workload, where the adaptation is based on mental processing capacity.

Their findings further highlight a broader issue: DDA implementation in serious games remains inconsistent, with most systems relying on one-measure approaches that work well in simpler environments but struggle in more complex ones. By contrast, multi-measure and multi-adaptation systems simulated stronger outcomes, especially when incorporating factors such as cognitive load and engagement, both of which are closely linked to learning success.

Seyderhelm and Blackmore thus argue that effective DDA in serious games must account for both performance and pedagogical demands, emphasising that future work should prioritise multi-measure triggers and more flexible adaptation strategies to support learning-focused game design better.

Extending this operational focus, more recent research has examined how adaptive principles are implemented within live game systems. Fisher and Kulshreshth (2024) organise this by identifying several algorithmic approaches used to modulate difficulty in contemporary titles:

- Fixed difficulty: Static difficulty that did not change throughout the investigation, used as a control.
- User selected: The player chose their preferred difficulty level, based on what the player was comfortable with.
- Performance-based: Metrics were derived from various in-game parameters. Various player inputs were measured and affected difficulty, including player health, last hit, and survival time.
- Emotion-based: In an attempt to maintain players' experiences within this optimal mental state described by Csikszentmihalyi (2008), EEG readings were taken over the course of the play session, and at periodic intervals, the difficulty would be changed based on measurements of stress and engagement.
- Combined approach: This method integrated elements of both performance-based and emotion-based metrics to determine the overall

difficulty of the experience, albeit weighted towards emotion-based metrics, as that was found to have more impact on players' enjoyment and immersion.

In the end, it was discovered that no DDA technique demonstrated clear superiority over another. DDA was not favoured over static settings. Adaptive difficulty was found to imply that more nuanced emotional and cognitive modelling is required.

2.4 Relevance

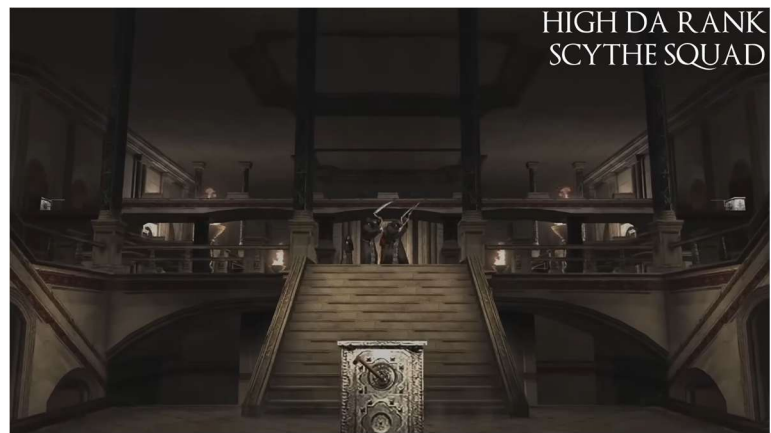
The limitations identified across both theoretical and applied research highlight a clear gap between the potential of adaptive systems and their current implementation in serious games and mainstream commercial titles. Although the literature demonstrates that Dynamic Difficulty Adjustment may preserve flow, support education, and mitigate the extremes of boredom and anxiety, contemporary systems remain inconsistent, often relying on narrow performance metrics or single-measure triggers. This inconsistency contrasts with the commercial games industry, where adaptive mechanisms have been refined over decades and now function as core components of player experience. Examining these real-world systems provides a valuable point of comparison, illustrating both the feasibility and the necessity of more sophisticated adaptive models. The following case studies: *Resident Evil 4* (Capcom, 2005) and *Celeste* (Mady Makes Games, 2018), demonstrate how contemporary games operationalise adaptation in practice, and where these approaches still fall short.

2.4.1 Case Study: *Resident Evil 4* (2005)

Capcom's *Resident Evil 4* (2005) remains one of the most widely examined examples of performance-based dynamic difficulty adjustment (DDA) in commercial game design. The game utilises a hidden "Dynamic Adjustment"

(DA) mechanic that evaluates the player's performance in real time and modifies certain internal gameplay parameters. This system, while never made obvious to the player, operates on a numerical value that shifts upward or downward depending on how well the player is progressing through encounters. As documented through community reverse-engineering and long-form analysis (Maxylobes, 2024), the system monitors variables such as damage taken, accuracy, enemy kills, and player deaths, aggregating these into a single internal score that governs enemy behaviour, item distribution, and encounter composition.

At higher DA levels, enemies become more aggressive, move faster, and deal more damage, while ammunition drops become noticeably scarcer. Conversely, when the DA score decreases, the game compensates by easing enemy pressure and increasing the likelihood of healing items or lower-tier ammunition (Maxylobes, 2024). This creates a fluid difficulty curve that adapts continuously to the player's performance, without the need for manual difficulty selection. For instance, in the "Water Hall" sequence, the number and types of enemies that spawn at the start of the room vary significantly based on the player's current DA level. Players doing well may face higher quantities of enemies, including additional ranged attackers, whereas players with a lower DA score encounter fewer threats (see Figure 2). This clearly shows how the same encounter can feel different for a player, depending on their prior performance.



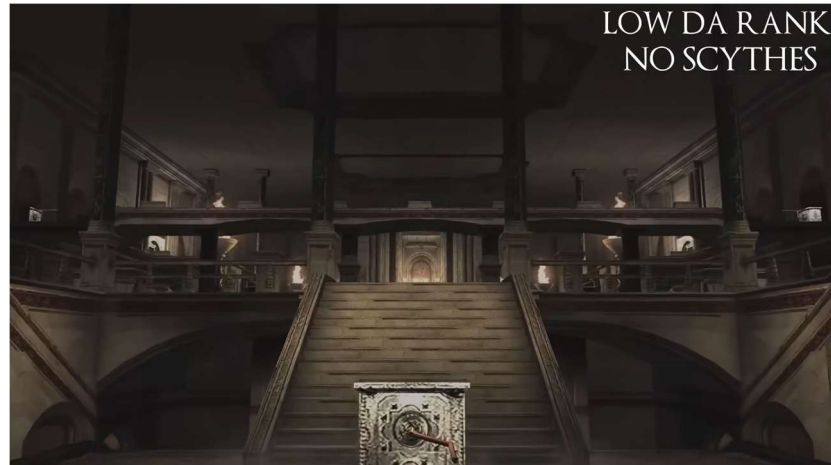


Figure 2. Water Room spawns at a High DA Rank.

Two crossbow-wielding enemies spawn on the walkway above (one on either side), and a squad of scythe-wielding enemies spawns after the staircase is lowered. By contrast, lowering the difficulty results in neither the crossbow-wielding enemies nor the squad of scythe-wielding enemies appearing. Source: Maxylobes (2024).

The DA system also influences resource management, a core tenet of survival-horror design. When players are low on health and lack healing items, the game increases the probability of healing item drops. When players are performing well, the system prioritises ammunition drops, thereby shoehorning players into a more aggressive playstyle (Maxylobes, 2024). This dynamic resource economy aims to prevent soft-locking and ensure that players who struggle are not permanently punished for earlier mistakes. It also maintains the series' characteristic tension curve, where scarcity and relief alternate in a controlled rhythm.

Despite its in-depth implementation, Resident Evil 4's DA system also highlights the limitations of performance-based adaptation. Because this system relies heavily on behavioural metrics, it does not account for the causes of poor

performance. Whether due to cognitive overload, emotional stress, unfamiliarity with controls, or psychomotor accessibility barriers, the system may misinterpret the player's internal state, thereby adjusting the difficulty in ways that do not necessarily support the player's engagement or flow. This limitation underscores the need for multi-measure adaptive systems that incorporate affective or cognitive indicators.

2.4.2 Case Study: Celeste

In contrast to Resident Evil 4's hidden, performance-based difficulty adjustment, 2018's *Celeste* (Maddy Makes Games Inc., 2018) adopts a completely different approach, using an explicitly accessibility-driven model. Through its player-modifiable "Assist Mode", the developers designed an interface that allows users to directly control specific challenge parameters rather than relying on automatic changes. By providing a suite of adaptive tools, including adjustable game speed, infinite stamina, air-dash limit changes, Dash Assist, and invincibility toggles, the game allows players to tailor its temporal, mechanical, and punitive demands to their own capabilities (see Figure 3).

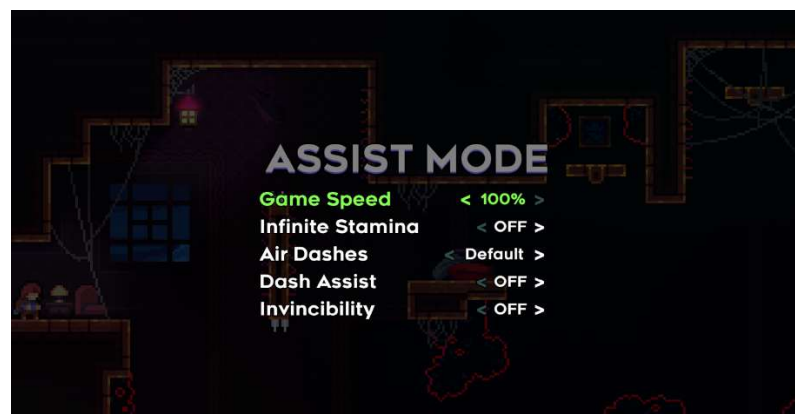


Figure 3. Assist mode options menu in *Celeste* (2018). Screenshot captured by the author.

Celeste's Assist Mode has the ability to mitigate potential temporal bottlenecks that may disrupt player flow in precision-platforming games. By allowing players to reduce the game's overall speed, the system effectively widens input windows and lowers the psychomotor demands associated with rapid directional changes, dash timing, and mid-air corrections. As Carvalho et al. (2024) note, adaptive tools like these are critical for systematically removing inherent game barriers, allowing notoriously difficult mechanics to become accessible to a wider audience. Features such as Dash Assist further support this by buffering inputs and reducing the precision required to execute complex manoeuvres. Ultimately, these tools directly address the temporal and motor-coordination barriers that disrupt flow. As Aytemiz et al. (2020) conclude in their specific analysis of *Celeste*, providing direct interventions to mitigate strict timing constraints is a crucial accessibility strategy that prevents players from becoming overwhelmed and forced to abandon the game.

The approach Maddy Makes Games takes to difficulty adjustment offers several notable benefits. The customisable nature of Assist Mode gives players full, direct control over their experience, enabling them to self-regulate flow by adjusting the challenge parameters to match their own skill level. This shift from fixed constraints to player-driven autonomy satisfies a core psychological need for self-determination, ensuring players remain intrinsically motivated even when facing difficult mechanical hurdles (Ryan, Rigby, and Przybylski, 2006). This autonomy supports the maintenance of the proportionality between challenge and skill that Csikszentmihalyi (2008) identifies as central to achieving flow. Furthermore, by allowing players to make proactive adjustments, as opposed to relying on hidden performance algorithms, the system helps prevent frustration, anxiety, or boredom arising from either excessive demands or insufficient challenge (Fisher and Kulshreshtha, 2025). In this model, difficulty

becomes a negotiable property rather than a fixed constraint, reflecting a broader accessible-design philosophy in which players can systematically tailor the game's demands to their individual capabilities (Carvalho et al., 2024).

However, Assist Mode isn't without its drawbacks. Relying on manual difficulty customisation assumes that players possess perfect self-awareness, specifically, the cognitive capacity to accurately recognise when they are struggling and why. This reliance on conscious self-evaluation is a notable limitation; as Nacke and Lindley (2008) argue, subjective reporting is inherently flawed because players are often unreliable judges of their own dynamic cognitive states during active gameplay. Furthermore, because the player must take retroactive, manual action to alter these settings, the system lacks the capacity for real-time adaptation. Navigating away from the game to adjust user interface options risks breaking immersion, thereby interrupting the very flow state the system is attempting to preserve, a disruption that Hunicke (2005) explicitly warns against, advocating instead for adaptive systems that abstract difficulty scaling away from conscious player control. Consequently, the risk for over- or under-reducing the challenge level is significant, undermining Csikszentmihalyi's (2008) principle of optimal difficulty. In the end, the lack of real-time measurement means that fluctuating variables such as frustration, cognitive load, and emotional state are entirely unaccounted for (Seyderhelm and Blackmore, 2021). While Assist Mode represents a substantial step forward for accessibility, it functions as a static toolset rather than a truly responsive system. This highlights a critical gap: the necessity for dynamic difficulty systems that adapt based on a continuous player state, rather than relying exclusively on retroactive player choice.

CHAPTER 3

NEW IDEAS

3.1 Key Research Focus

The analysis presented earlier highlighted a persistent gap between existing accessibility tools and adaptive systems designed to support player engagement. Manual accessibility settings depend on the player's ability to recognise and articulate their own psychomotor or processing limitations (Nacke and Lindley, 2008), whereas performance-based Dynamic Difficulty Adjustment (DDA) systems respond only to observable behaviour, overlooking the cognitive or affective causes of struggle (Seyderhelm and Blackmore, 2021). As a result, neither approach reliably maintains the challenge-skill balance required for flow, particularly for players whose impairments affect reaction time, motor coordination, or processing speed (Csikszentmihalyi, 2008).

The central aim of this investigation is to examine whether a performance-based adaptive system can effectively support these players by dynamically adjusting mechanical challenge in response to in-game behaviour. To assess whether the system maintains proportionality between challenge and skill, the project incorporates EEG-derived engagement measures as an external indicator of flow state (McMahan, Parberry, and Parsons, 2015). In this model, physiological data do not drive adaptation; instead, they provide an independent measure of cognitive engagement to evaluate the effectiveness of the performance-based approach.

The work, therefore, focuses on designing and implementing a prototype that applies performance-based difficulty adjustments while simultaneously recording

EEG data to track changes in engagement. This dual-layer approach enables an examination of whether performance-based adaptation alone is sufficient to support players with psychomotor or processing impairments, and whether it aligns with the theoretical requirements for sustaining flow.

3.2 Prototype Design and Functional Requirements

3.2.1 Prototype Overview

The design of the prototype is directly informed by the limitations identified in the preceding contextual analysis. Because psychomotor and processing impairments can easily push a player out of the optimal flow channel due to rigid mechanical timing, a more responsive intervention is required. Therefore, this prototype was purpose-built as an investigative testbed to evaluate the efficacy of autonomous, performance-based Dynamic Difficulty Adjustment (DDA). Rather than functioning as a standard commercial product, the game environment was strictly controlled to isolate the variables of challenge and skill. This structured approach guarantees that any resulting physiological data is a direct reaction to the DDA's deliberate adjustments, rather than arbitrary game design.

3.2.2 Prototype Design

To effectively evaluate the impact of Dynamic Difficulty Adjustment (DDA) on psychomotor accessibility, the prototype required a gameplay loop that actively tested a player's reaction speed, motor coordination, and sensory processing capacity. Consequently, the prototype was designed as a high-density, third-person action boss encounter. Unlike puzzle or strategy games, action-oriented games force players to make split-second inputs within strict time frames, making them an ideal environment for inducing and measuring cognitive friction and flow state disruption.

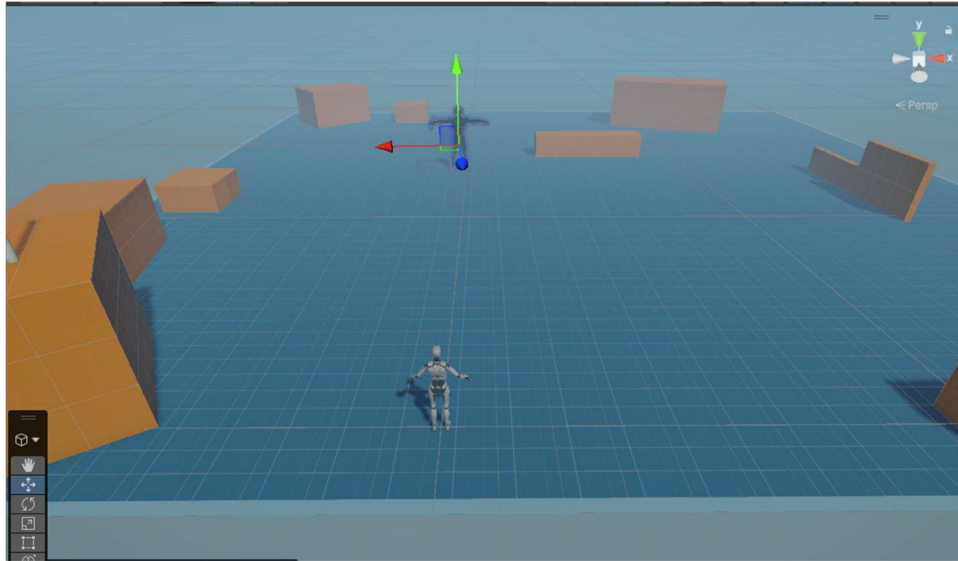


Figure 4. Overview of the testing environment built using Unity Starter Assets.

The encounter was deliberately structured into two combat phases to facilitate data gathering:

- Phase 1: The initial phase of the encounter is designed with standard attack telegraphing and predictable enemy pacing. This phase serves as a period for the player to familiarise themselves with the enemy's attack pattern. It also allows `DdaInputTracker.cs` to establish a performance average, specifically recording the player's raw reaction times and dodge success rates over the initial 35 seconds of combat.
- Phase 2: Triggered when the boss's health is reduced to 50%, this phase introduces an intentional spike in difficulty. The encounter becomes more difficult through a visual signifier of the transition to Phase 2, an increase in a static damage buff, the enemy healing some health, and the unlocking of a broader repertoire of more complex attack patterns. This was intended to push the player towards the "anxiety" threshold of the Flow Channel, fostering a situation for the DDA to intervene.



Figure 5. The enemy executing a standard Phase 1 attack pattern, establishing the baseline mechanical friction prior to the intervention of the Fuzzy Logic controller.

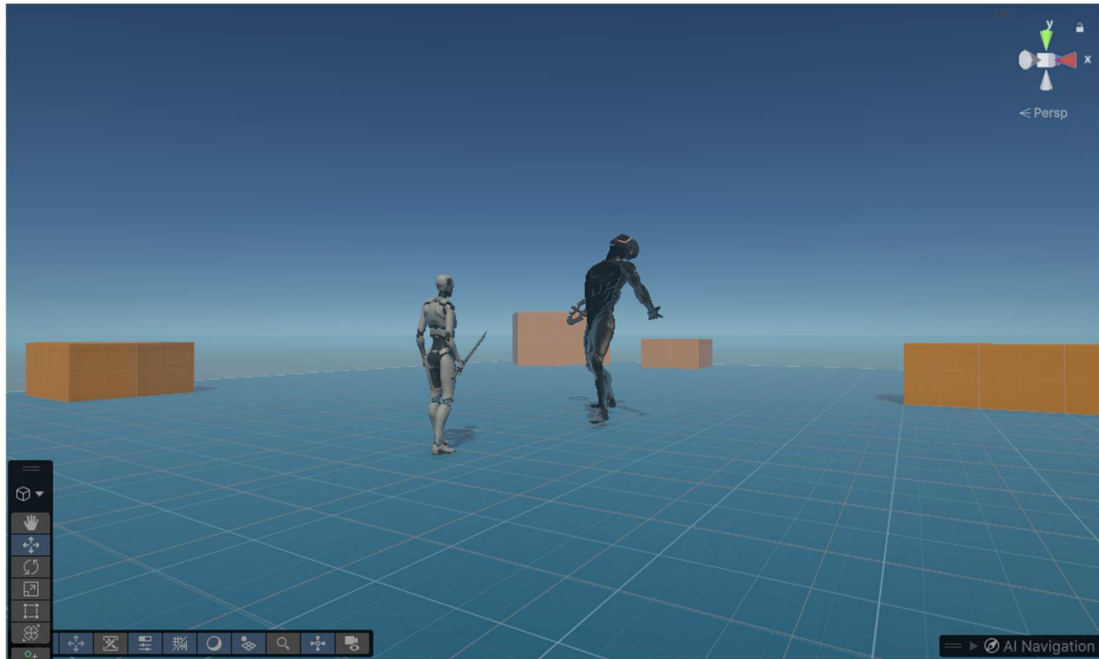


Figure 6. The adversary executing a 'battlecry' state animation, visually signifying the transition into the higher-difficulty second phase of the encounter.

To further ensure the validity of telemetry, the environmental and user interface (UI) design was guided by established accessibility and human-computer interaction (HCI) heuristics. Specifically, the prototype's visual feedback loop adhered to the core cognitive framework of the Game Accessibility Guidelines (GAG), which requires that "[e]nsure that all important supplementary information (eg. the direction you are being shot from) conveyed by audio is replicated in text / visuals" (Game Accessibility Guidelines, 2024). Furthermore, the UI was intentionally kept minimalist and simple (see Figure 7). Lastly, the game supports mouse, keyboard, and controller inputs, again in line with the GAG: "Support more than one input device".

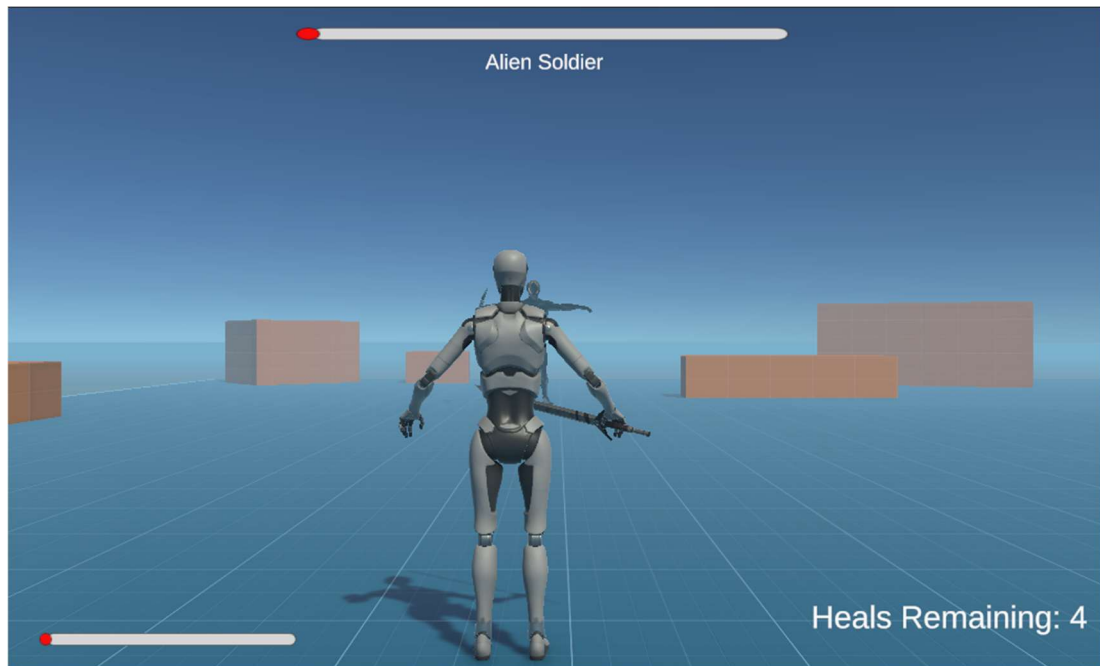


Figure 7. Player perspective during the prototype experience, with a minimalist User Interface.

3.2.3 Fuzzy Logic Controller

To translate the theoretical need for adaptation into a functional software architecture, the prototype's artificial intelligence was designed using the six-step operational workflow proposed by Hendrix et al. (2019). This framework requires that a successful Dynamic Difficulty Adjustment system systematically operationalise the player's performance. For this prototype, the workflow was implemented as follows:

1. Identify indicators of player performance: track input variables in real time, in this study's case: 'reaction speed' and 'success rate'. These refer to the time in between the start of an enemy's attack and the player's reactive input, and of the previous 10 attacks, what percentage were successfully dodged compared to attacks where damage was dealt.

2. Determine dynamic difficulty variables: scale the boss's output variables, specifically attack animation speed and attack damage multiplier.
3. Locate variable representation: Integrating these metrics directly into the core gameplay loop and the boss's state machine.
4. Identify affected mechanics: Modulating the intensity of the static damage buff introduced in Phase 2.
5. Define the calculation of difficulty: Utilising a continuous mathematical algorithm to scale the challenge based on real-time inputs.
6. Establish baseline values: Utilise the first 35 seconds of phase 1 of the prototype to establish an active, in-game performance baseline before introducing complex modifiers.

While Hendrix et al. provide the structural blueprint, their fifth step, which defines exactly how to calculate the difficulty, requires a robust algorithmic solution. Traditional binary DDA systems often rely on rigid numerical thresholds (e.g. IF health < 30, THEN difficulty = Easy). These binary triggers can cause jarring, abrupt shifts in gameplay that inadvertently disrupt any flow state being attempted to preserve.

To address this, the prototype fulfils Hendrix's fifth step by implementing a fuzzy logic controller, as advocated by Pirovano (2012). Unlike rigid binary systems, fuzzy logic offers a smoother, more human-like approach to reasoning. It does so by mapping the identified performance metrics to linguistic variables rather than strict numerical absolutes.

3.2.4 Requirements Analysis

Functional Requirements

Process-Oriented Requirements:

- The system must be consistently monitoring and evaluating real-time player telemetry and changing the boss's attack speed and damage multipliers accordingly and autonomously.
- The transition between states, specifically between Attacking Phase 1 and 2, must be smooth and trigger correctly. The static damage increase must be applied without interrupting the game loop.
- The multipliers calculated by the system must sufficiently reflect the change in difficulty so that it is noticeable to the player.
- TODO: Synchronisation requirement

Information-Oriented Requirements:

- The system must accurately capture, calculate, and temporarily store continuous gameplay data, specifically focussing on the input variables of 'Reaction Speed' and 'Success Rate'.
- The AI engine must routinely be fed the raw numerical telemetric data, map it to predefined linguistic variables (e.g. Low, Medium, High), and evaluate it against the hardcoded fuzzy rule matrix to output a definitive difficulty parameter to the boss behaviour tree.

Non-Functional Requirements

Operational Requirements:

- The User Interface (UI) must remain strictly minimalist, restricted only to essential feedback mechanics (health bars, heal counter). The software must not render complex menus, flashing elements or prolonged particle-heavy UI elements that could spike the user's cognitive load or the game's frame rate.

Performance Requirements:

- The Unity build must maintain a locked, stable 60 Frames Per Second (FPS). The codebase must be optimised to ensure there is no rendering lag, garbage-collection spikes, or input latency, as software-induced temporal bottlenecks would invalidate the performance data.

Security Requirements:

- The application must not initiate external network requests, use network-based APIs, or transmit data over the internet. All telemetry logs generated at runtime must be stored securely and locally.

Cultural and Political Requirements:

- The system must adhere to inclusive-design principles. The adaptive algorithm must serve as an invisible assistive intervention rather than a punitive difficulty slider. Supporting players who struggle with gameplay can preserve the “magic circle” as an accessible, autotelic space for all users.

3.3 Methodology

3.3.1 Apparatus and Instrumentation

In order to collect both physiological and subjective data, the experiment uses a mixed-methods approach.

- The OpenBCI Galea headset will be used to record real-time physiological data. This device allows the capture of electroencephalography (EEG) to monitor brainwave frequencies.
- Once the gameplay has concluded, the participant will complete the Flow Short Scale (FSS) (Rheinberg et al., 2023). This standardised questionnaire acts as the qualitative anchor to verify if the physiological

readings align with the user's self-reported feelings of "absorption" and "fluency" in the game. High scores on the FSS will indicate a successful manifestation of the flow state.

3.3.2 Experimental Procedure

The testing environment is designed to minimise external stimuli and cognitive noise. The procedure follows a strict chronological protocol:

1. Baseline Calibration (2 minutes). The participant is seated and asked to remain in a resting state (eyes open, staring at a static screen) for two minutes. This establishes a normative physiological baseline unique to the individual.
2. Active Gameplay (10 minutes). The participant engages with the two-phase prototype encounter. The system logs in-game telemetry (reaction speed and success rate) alongside the Galea headset's logic biometric responses.
3. Qualitative Assessment. Immediately after the gameplay session, the participant will complete a short survey, the Flow Short Scale (FSS), before any post-game discussion, to prevent memory decay or interviewer bias.

3.3.3 Data Analysis and The Engagement Index

To quantify the player's cognitive state in real time, the raw electrical signals captured by the OpenBCI Galea are mathematically decomposed into unique frequency bands. Within the context of this investigation, the primary focus is on Theta (θ), Alpha (α), and Beta (β) rhythms, as these bands react dynamically to varying stimulus modalities during gameplay.

The main metric for evaluating the efficacy of the Dynamic Difficulty Adjustment (DDA) is the Engagement Index (EI). Originally developed to evaluate task engagement in automated environments (Pope, Bogart, and Bartolome, 1995, as cited in McMahan, Parberry and Parsons, 2015), the index is calculated as follows:

$$EI = \frac{\beta}{\alpha + \theta}$$

During states of immersion, cortical activation suppresses alpha activity, whereas the heavy allocation of working memory and sustained attention drives an increase in frontal Theta power (Katahira et al., 2018). The mechanics of this formula dictate that as a player's active concentration increases, Beta power rises while Alpha and Theta power are suppressed, resulting in a higher overall EI score.

These psychological fluctuations are mapped directly onto the theoretical boundaries of Csikszentmihalyi's Flow Channel (2008). The "Flow State" is not represented by maximum EI values, which typically correlate with negative valence, hyper-arousal, or anxiety (Nacke, Stellmach, and Lindley, 2011). Instead, by monitoring the EI, the player's movement between three cognitive states can be observed objectively:

- Boredom (under-stimulation): The system challenge is too low to require active processing.
- The Flow Channel (optimal management): A sustained EI plateau where the game's mechanical friction perfectly matches the participant's psychomotor ability.

- Anxiety (hyper-arousal): A sharp spike above the upper EI threshold, signalling that the mechanical demands have surpassed the player's processing or physical capacity, breaking their immersion.

Importantly, these thresholds are not static, universal integers. Because baseline electrocortical activity varies significantly between individuals, particularly in neurodivergent participants, all boundaries must be calculated and standardised to each participant to account for physiological variance (Fairclough, 2009).

3.3.4 Mitigation of Extraneous Variables

Given the neurological and physical diversity of potential participants, several extraneous variables must be accounted for during data analysis:

Research by Arns et al. (2013) and Monastra et al. (1999) indicates that individuals with ADHD may exhibit an inverted resting baseline, characterised by higher frontal Theta and lower frontal Beta waves. To prevent this from skewing the Engagement Index, all active gameplay data will be calculated as a delta (Δ) relative to the individual's 2-minute resting baseline, rather than compared with a universal neurotypical average.

There is a risk that the Fuzzy Logic DDA may misinterpret delayed inputs caused by motor impairments as a simple lack of mechanical skill. If the algorithm lowers the difficulty in response, the EEG may drop below the "boredom" threshold, or, conversely, spike into "anxiety" due to the player's physical inability to execute intended actions. Evaluating the FSS survey alongside the EEG data will be critical to identifying these false positives.

3.4 Achieving Project Aims

The experimental methodology is designed to act as a traceability framework, mapping the practical testing directly onto the project's aims. Linking the two parts together, allows the methodology to rationalise the theoretical goals into measurable outcomes:

- Aim 1: Develop an autonomous Dynamic Difficulty Adjustment (DDA) system capable of interpreting player performance without explicit UI interaction.
 - o This is achieved through the implementation of the Fuzzy Logic Controller. DdaInputTracker.cs is integrated to routinely poll raw data from the player (Reaction Speed and Success Rate) and feed it into the Accord.Fuzzy inference engine, which evaluates the player's performance in the background. The "magic circle" is better preserved when the responsive system requires no manual inputs.
- Aim 2: Objectively validate the efficacy of the DDA system in sustaining a player's Flow State using neurophysiological biometric tracking.
 - o This is addressed through the mixed-methods testing protocol being implemented. Rather than relying exclusively on subjective player feedback, which may be unreliable due to poor self-awareness of cognitive states, the OpenBCI Galea headset will be used to provide real-time biometric data. Calculating the Engagement Index (EI) from Theta, Alpha, and Beta provides an objective metric to demonstrate the DDA's effectiveness in adjusting the player's cognitive arousal.

3.5 Methodology Critique

While the selected methodology provides a robust framework for conducting the investigation, there are some trade-offs in the experimental design.

Primarily, the main critique stems from the tension between internal and ecological validity. To ensure that the EEG readings were a result of mechanical friction, the prototype was designed as a highly controlled, minimalist testing environment stripped of any complex game mechanics and design. This strict environmental control successfully isolates the independent variables, but it significantly lowers the ecological validity of the study. A commercial video game features a mixture of complex sensory inputs, overlapping audio, and complex visuals. As such, this study may not be representative of a full-scale product without further multidimensional testing.

Secondly, the physical nature of the apparatus may cause some artificial performance anxiety. This may stem from the prospect of wearing a heavy headset, which is uncomfortably placed on the head as it reads brain activity, or from the fact that the study occurred in a laboratory. While measuring the delta (Δ) against each individual baseline removes the impact of the anxiety, it may still persist and, in turn, affect the data.

CHAPTER 4

INVESTIGATION

4.1 Project Planning and Methodology

4.1.1 Development Methodology

The development occurred in two main phases. The first phase occurred within Term 1 to the beginning of Term 2. Phase two occurred towards the end of Term 2 and Term 3.

The first phase was less stringent in terms of development deadlines, and thus a less strict-on-time methodology was used in the form of incremental development. Breaking the prototype down into segments, and working on each one was done iteratively, but linear in the sense of moving from one segment to another.

The second phase was more short on time, provided it occurred at a time when other assignments' deadlines were very close together, thus required more focus overall than the prototype at the time. The methodology used in the second phase was chosen to reflect the limited time frame, and as such it was decided that an Agile-Scrum development style was most appropriate (Schwaber and Sutherland, 2020), however sticking to the incremental and iterative development cycle as before.

Once the prototype was in a completely implemented state, testing went in and retroactively resolved using regression testing, going back and checking that code modifications aren't harming existing functionality or introducing new bugs.

4.1.1 Timeline and Workload Management

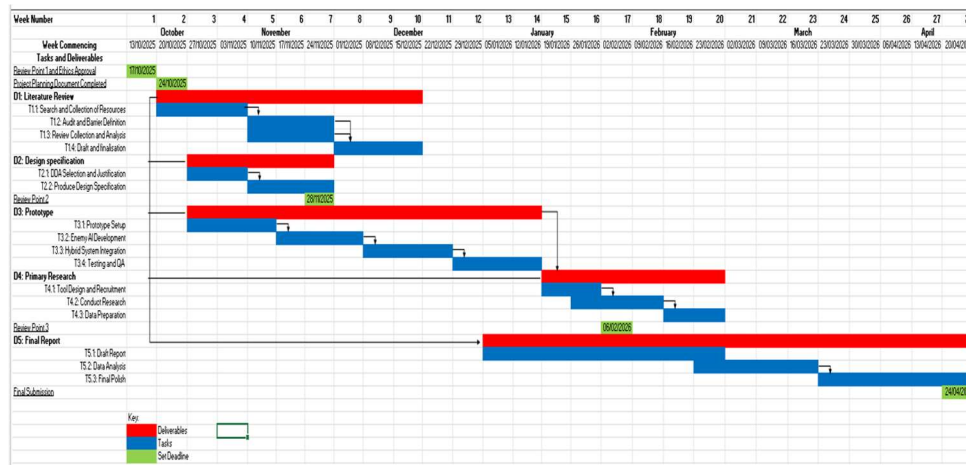


Figure 8. The original GANTT chart.

The timeline for this project ended up looking substantially different from the original plan (see Figure 8). While the original plan assumed a linear progression of tasks across the academic year, the reality of the development cycle required significant structural changes.

The initial phases of the project adhered closely to the proposed schedule. The research and basic literature review, primary research design, and the early stages of prototype development were all on schedule.

However, as the project transitioned into Term 2, a cluster of concurrent academic deadlines across other modules required more attention at that time. Because these external deadlines required immediate attention, the project schedule was forced to compress. Therefore, the bulk of the complex prototype development, especially the state machine logic and the Fuzzy Logic integration with Accord.Fuzzy, along with the formal dissertation write-up, were heavily backloaded towards the final weeks of the development period.

As discussed in Section 4.1.1, the inevitable schedule compression was precisely what necessitated the pivot from an incremental methodology to an Agile-Scrum methodology. By managing the project scope and prioritising core biometric tracking data over aesthetic polish, the critical technical deliverables were met despite the heavily restricted timeframe.

4.1.2 Risk Management and Contingencies

Given the complex setup of the investigation, the risks presented would be more focused on engineering and methodological risks. The main risks identified and managed during the development lifecycle included:

- Given the project's reliance on external frameworks, the architecture was highly vulnerable to package conflicts. The way around this was strict adherence to Git and Unity Version Control. This proved extremely important when the project engine migrated from Unity 6.2 to 6.3. This caused the physics to break within the prototype; however, it was able to be rectified using the version control available in the cloud.
- Concurrent academic deadlines compressed the available development window across Terms 2 and 3, so there was a high risk of failing to deliver the promised mechanics. To circumvent this, the development methodology had to shift to Agile-Scrum, with a focus on backend programming rather than front-end development or game design.

4.2 System Architecture and Software Design

4.2.1 High-Level Technologies

The prototype's architecture integrates three core components: Unity, the Galea EEG headset, and the Accord.NET libraries. Unity handled all gameplay logic, C# compilation and execution, and real-time psychomotor interactions. The Galea

hardware and GUI were used to record raw EEG signals during each session, which were exported directly as .txt files. These files were then processed after the session using Galea Tools (Buxton, 2026) to compute the Power Spectral Density (PSD) and Engagement Index (EI). Within Unity, the Accord.Math and Accord.Fuzzy libraries were used to perform fuzzy logic calculations locally, ensuring all difficulty adjustments occurred client-side without reliance on external services or network communication

4.2.2 Object-Oriented Design (OOD)

Due to the nature of Unity and other game engines, programming works on an Object-Oriented basis. Each object within a scene contains components, of which some of those are built-in functionality i.e. character controller, physics-based collision boxes i.e. colliders, and scripts. These scripts typically inherit from an abstract class provided by Unity called MonoBehaviour. Therefore, specific scripts are set to specific objects to ensure smooth functionality. This includes attaching the player movement script to the player avatar, or attaching the fuzzy brain logic to the enemy, who needs live access to the inputs provided by the DDA controller.

4.2.3 HCI and Usability Design

As outlined in Section 3.2.2, the Human-Computer Interaction (HCI) design centred on reducing cognitive load. The user interface (UI) was intentionally kept minimal, following the established Game Accessibility Guidelines (2024), so that nothing on-screen competed for the player's attention. By removing unnecessary menus, flashing indicators, and decorative feedback, the interface ensured that any changes in EEG activity were driven by the player's interaction with the combat system rather than by visual distractions. This helped protect

the validity of the OpenBCI recordings by keeping visual noise to an absolute minimum.

4.3 Prototype Development

4.3.1 Core Mechanics and Environment

The testing arena was built using a heavily modified version of the 'Third Person Character Starter Assets' package (Unity Technologies, 2024), sourced from the Unity Asset Store. The package also provided the character model, basic locomotion, and a camera setup, all of which were used.

The player controller utilised a third-person input layout, with inputs mapped to both the controller and the mouse and keyboard. Certain actions are tied directly to the DDA algorithm:

- Standard omni-directional movement using Unity's built-in Input System, provided by the Unity Starter Assets.
- A loosely time-based dodge mechanic that would trigger the input tracking for the player's reaction speed. Within the dodge, a few moments of invincibility frames (I-frames) were implemented.
- A basic, responsive melee attack sequence. This is the primary way the player would interact with the enemy and deal damage.
- A generous blocking action was implemented that when pressed and the enemy was attacking; the damage would be negated. This also triggered the script tracking the player's reaction speed.
- A limited healing ability was provided, allowing the player to restore 33% of their health 4 times during the encounter. This would help the player's

survivability, keeping them alive and giving them a better chance of defeating the boss.

4.3.2 Enemy AI and State Machine

The boss's behavioural logic in the prototype is governed by a Finite State Machine (FSM). To maintain codebase modularity and adhere to Object-Oriented Design principles, the foundational structure was adapted from Weimann's (2020) interface-driven state pattern. While this structure provided a stable base for state encapsulation, the core codebase was greatly expanded to support the multi-phase, dynamic encounter required for the DDA methodology.

Instead of relying on rigid conditional statements, the architecture utilises discrete state classes that implement a unified IState interface:

```
public interface IState
{
    // Code taken and adapted from Jason Weimann's "Unity Bots with
    // State Machines - Extensible State Machine / FSM" video, available on
    // YouTube: https://www.youtube.com/watch?v=V75hgcsCGOM

    void Tick();
    void OnEnter();
    void OnExit();
}
```

Within the EnemyStateMachine.cs component, the FSM and all associated states are instantiated during the Awake() method. This method establishes all transitions between states and their associated triggers, continuously monitoring which state the enemy should be actively using in that current moment. When a transition condition is met, the FSM calls Exit() on the current state, and Enter() on the newly designated state. This makes sure enemy AI is decoupled allowing the Fuzzy Logic to function correctly without breaking the overarching loop.

```

private void Awake()
{
    player = GameObject.FindGameObjectWithTag("Player").transform;

    _agent = GetComponent<NavMeshAgent>();
    _agent.updateRotation = false;

    _animator = GetComponent<Animator>();
    var playerDetector = GetComponentInChildren<PlayerDetector>();

    _health = GetComponent<Health>();

    _stateMachine = new StateMachine();

    var patrol = new PatrolState(this, _agent);
    var chase = new ChasePlayerState(this, _agent, player);
    var attack = new AttackState(this, _agent, player, _animator,
axe, foot);
    var death = new DeathState(_animator, _agent);

    var phase2Transition = new PhaseTransitionState(this, _animator,
_agent, _health);

    At(patrol, chase, () => DistanceToPlayer() <= detectionRadius);
    At(chase, patrol, () => DistanceToPlayer() > detectionRadius);

    At(chase, attack, () => DistanceToPlayer() <= attackRange);

    At(attack, chase, () => !isAttacking);
    At(attack, patrol, () => !isAttacking
&& !playerDetector.PlayerInRange);

    _stateMachine.AddAnyTransition(death, () => isPhase2 && isDead);
    _stateMachine.AddAnyTransition(phase2Transition, () =>
_health != null && _health.currentHealth > 0 &&
_health.currentHealth <= (_health.maxHealth * 0.5f) && !isPhase2
&& !isDead);

    At(phase2Transition, chase, () => phase2Transition.IsComplete);

    _stateMachine.SetState(patrol);

    void At(IState from, IState to, Func<bool> condition) =>
_stateMachine.AddTransition(from, to, condition);
}

```

The FSM is comprised of five primary behavioural states, designed to provide different parts of the enemy behaviour:

- Patrol State: The default, low arousal state. The enemy navigates to a random point with a preset detection radius, which it will move to using

Unity's NavMesh system. This adds a layer of realism to the boss fight, allow the player to have a moment to settle into the experience.

- Chase State: The player has moved into the enemy's detection radius. The AI updates the NavMesh destination to the location of the player, following and approaching the player.
- Attack State: Once the player falls within the enemy's attack range. The amount of attacks the enemy can use varies dependent on the current combat phase the enemy is in. Phase 1 has 3 attacks, whereas phase 2 has 6.
- Phase 2 Transition State: When the enemy's health reaches below 50% of its maximum, the combat loop is halted temporarily. A brief animation plays and phase 2 begins. The enemy regains some health, and gains access to some extra attacks.
- Death State: The final state triggered when the enemy's health pool reaches 0. The FSM is stopped, and the

4.3.3 Implementing the Fuzzy Logic Controller

To program the DDA algorithm design into a functional Unity component, the project utilises the Accord.Fuzzy framework (Souza et al., 2014). This open-source namespace performs the heavy lifting for the FuzzyBrain.cs controller. By offloading complex, low-level algorithms, such as calculating intersecting memberships and resolving output matrices, to a third-party library such as this, development resources were more focused on developing the behavioural rulesets pertaining to the boss's AI.

The implementation of fuzzy inference system follows three stages: fuzzification, rule database formulation, and defuzzification.

In order for the engine to be able to make decisions, it must convert raw telemetric data from the game and convert it into a format the fuzzy logic

controller can understand. The system translates absolute values into overlapping linguistic states. The following snippet demonstrates the instantiation of the 'Damage Multiplier' variable, applying trapezoidal membership functions to define Low, Normal, and High states:

```
LinguisticVariable dmgMultiplier = new
LinguisticVariable("DamageMultiplier", 0.5f, 1.5f);
dmgMultiplier.AddLabel(new FuzzySet("Low", new
TrapezoidalFunction(0.5f, 0.5f, 0.7f, 0.9f)));
dmgMultiplier.AddLabel(new FuzzySet("Normal", new
TrapezoidalFunction(0.8f, 0.9f, 1.1f, 1.2f)));
dmgMultiplier.AddLabel(new FuzzySet("High", new
TrapezoidalFunction(1.1f, 1.3f, 1.5f, 1.5f)));
```

As demonstrated above, the architecture strictly enforces overlap. This prevents 'dead' zones that may cause rigid state machines to fail.

With all variables and memberships functions defined, the next step would be establish logic matrix to dictate how the inputs interact and output a definitive state for the boss.

```
Database db = new Database();
db.AddVariable(reactionSpeed);
db.AddVariable(successRate);
db.AddVariable(animSpeed);
db.AddVariable(dmgMultiplier);

engine = new InferenceSystem(db, new CentroidDefuzzifier(1000));

// Rules for low reaction speed (see FuzzyBrain.cs in the Project
Files)

// Rules for average reaction speed
engine.NewRule("R_AverageButLow_AnimSpeed",
    "IF ReactionSpeed IS Average AND SuccessRate IS Low THEN
AnimationSpeed IS Normal");
engine.NewRule("R_AverageButLow_DmgMultiplier",
    "IF ReactionSpeed IS Average AND SuccessRate IS Low THEN
DamageMultiplier IS Low");

engine.NewRule("R_AverageButMedium_AnimSpeed",
    "IF ReactionSpeed IS Average AND SuccessRate IS Medium THEN
AnimationSpeed IS Normal");
engine.NewRule("R_AverageButMedium_DmgMultiplier",
    "IF ReactionSpeed IS Average AND SuccessRate IS Medium THEN
DamageMultiplier IS Normal");

engine.NewRule("R_AverageButHigh_AnimSpeed",
```

```

    "IF ReactionSpeed IS Average AND SuccessRate IS High THEN
    AnimationSpeed IS Normal");
engine.NewRule("R_AverageButHigh_DmgMultiplier",
    "IF ReactionSpeed IS Average AND SuccessRate IS High THEN
    DamageMultiplier IS High");

```

These rules act as the core decision-making parameters. By utilising fuzzy logic AND operators, it allows multiple factors to be considered, thus alleviating the risk of conflicting measurements.

Finally, during active gameplay, the DdaInputTracker.cs (See Appendix A) feeds real-time information into the above established matrix. The Accord.Fuzzy engine evaluates which rules fire and aggregates the overlapping output states. Finally, it performs Centroid Defuzzification: calculating the exact mathematical centre of mass of the resulting shape to return a single float value. This float value is updated periodically, every 35 seconds period. The script collects the outcome of the last 5 measurements of reaction speed and the outcome of the previous 10 attacks of the boss to monitor which ones dealt damage. This information was stored in a queue for each input and a percentage of successful to unsuccessful attacks dodged or blocked.

```

public void EvaluatePlayer(float currentReaction, float
currentSuccess)
{
    currentReaction = Mathf.Clamp(currentReaction, 0f, 2.0f);
    currentSuccess = Mathf.Clamp(currentSuccess, 0f, 100f);

    engine.SetInput("ReactionSpeed", currentReaction);
    engine.SetInput("SuccessRate", currentSuccess);

    float animSpeedMultiplier = engine.Evaluate("AnimationSpeed");
    float damageMultiplier = engine.Evaluate("DamageMultiplier");

    if (animSpeedMultiplier == 0) animSpeedMultiplier = 1.0f;
    if (damageMultiplier == 0) damageMultiplier = 1.0f;

    animSpeedMultiplier = Mathf.Clamp(animSpeedMultiplier, 0.7f,
1.2f);
    damageMultiplier = Mathf.Clamp(damageMultiplier, 0.5f, 1.5f);

    calculatedAnimSpeed = baseEnemySpeed * animSpeedMultiplier;
    calculatedDamageMod = baseEnemyDamage * damageMultiplier;

    if (_stateMachine != null)

```

```

{
    _stateMachine.currentDdaSpeed = calculatedAnimSpeed;

    Animator anim = _stateMachine.GetComponent<Animator>();
    if (anim != null) anim.speed = calculatedAnimSpeed;

    if (_stateMachine.isPhase2)
    {
        calculatedDamageMod *=
        _stateMachine.phase2DamageMultiplier;
    }

    if (_stateMachine.axe != null)
    {
        _stateMachine.axe.damageAmount = (int)
        calculatedDamageMod;
    }

    if (_stateMachine.foot != null)
    {
        _stateMachine.foot.damageAmount = (int)
        (calculatedDamageMod / 2);
    }
}

```

This returns the animation speed and damage multipliers that are passed directly into the Finite State Machine, thereby altering the relevant in-game factors. This allows the system to analyse and adjust the game's difficulty, unbeknownst to the player, not requiring any manual input.

4.4 Software Testing and Verification

4.4.1 Testing against requirements

After the prototype met the structural specifications in Section 3.2.4, a functional testing protocol was applied before user trials. Process-oriented requirements were validated through active runtime debugging, confirming that the Accord.Fuzzy engine successfully executed dynamic adjustments without interrupting the boss's EnemyStateMachine.

Information-oriented requirements were met by confirming that the input data was stored in localised queues to calculate moving averages before passing it to the inference engine.

Finally, non-functional performance and security requirements were confirmed by a stable frame rate, and the network architecture was strictly local, so no external APIs were used.

4.4.2 System Testing

Once the system was in a fully implemented state, the prototype was tested to detect any malfunctions. Multiple errors and bugs were identified. Below are some that were encountered and whether they were deemed necessary:

- If an animation would move the enemy from its root, and the exit transition would always return it to the root, making the animation look less fluid. This required converting all animations to humanoid-copy from other avatar-bake into pose. Further testing still did not work, so they were replaced with static attack animations.
- There were occasions when errors occurred with membership functions equating to 0. This was fixed by clamping the success rate between 0 and 100, preventing NaN values.
- The enemy didn't always face the player, so it was easier for the enemy to miss attacks, potentially skewing the success rate. This was alleviated by implementing a FacePlayer() function, which calculates the normalised direction from the boss to the player and then faces that direction.
- The original implementation of the FacePlayer() function faced 45° counterclockwise from the player's actual direction. This was accounted for in the Quaternion value for the Slerp by adding a Euler multiplier of (0, 45f, 0) to the base rotation.
- Animations are not always stationary. For example, the animation signifying the transition into phase 2 continued to follow the player even when the player

was in a seemingly stationary pose. This bug was stubborn and deemed non-problematic, so it was not fixed.

- If the player initiated a dodge towards the enemy, the colliders would collide incorrectly, causing the player to dodge over the enemy. Additionally, the player could stand above the enemy. Whilst not an ideal bug, it was ultimately deemed unproblematic and left unfixed.

- Another cause of the error, which resulted in the membership functions crashing due to extraneous inputs, was that the reaction speed variable fed into the system exceeded the limits of the Fuzzy Sets within the linguistic variable. This was rectified by clamping the variable fed into the system and by implementing an averaging system, so the 5 most recent reaction speeds were recorded in a Queue<float>, and the average for that was taken to provide more accurate data to the controller.

- There are some bugs in the transitions within the enemy's locomotion controller. Although a minor issue, it was deemed unproblematic and not worth rectifying.

4.5 Development Process Evaluation

- Although the development process itself was carried out over several months, it was delayed by other coursework and software development projects, so the prototype took longer than anticipated. This resulted in a somewhat SCRUM-esque development sprint, in which the remaining sections of the programme were developed within a limited period of time. As a result, there were some issues with regards to bugs and issues that were higher prioritised than others, resulting in certain bugs to remain unresolved.

4.6 Carrying out the investigation

4.6.1 Setup

First and foremost, an information brief and consent form were put together to provide prospective participants with as much information as possible, to give them the best chance to give informed consent. These were documents that laid out the purpose of the study, how it would be carried out, any risks and benefits, and valuable information including contact details and data protection information, as all data is stored in compliance with General Data Protection Regulation (GDPR) as per the Data Protection Act 2018

Participants were approached personally, and all volunteers were classmates and connections within social networks. In total, 6 participants expressed interest, but only 3 were available to take part in the pilot study. Of these, 2 (67%) identified as female and 1 (33%) identified as male. They were also asked whether they identified as neurodivergent and, if so, which diagnoses. 1 participant reported no neurodivergence, whereas 2 reported Autism Spectrum Disorder (ASD) and Attention Deficit Hyperactivity Disorder (ADHD), and 1 also noted Dyslexia. Finally, they were asked whether they had been diagnosed with or self-reported any conditions that may affect processing speed, reaction time, or motor coordination. Only 1 reported a related diagnosis, namely an Anxiety Disorder.

The Galea GUI program collecting and recording the telemetric data was connected to a wired PC, whereas the game itself was setup on a personal laptop, so no Unity SDK linking the two programs was used. The participant would be using mouse-and-keyboard to interact with the game. A note file was created containing all necessary controls was displayed to the participant for the duration of the activity portion of the study.

On the day of the pilot study, the academic associates supervising provided a brief, explaining how the software operating, how to appropriately fit the headset on someone, basic troubleshooting protocols and other important base information.

The main GUI used for recording data was Galea GUI, which was connected to the Galea headset via Bluetooth. The data was compiled using BrainFlow SDK and would be output as a .csv and a .txt in CSV format.

4.6.2 Execution

Each participant was asked to confirm if they had read the study brief and had filled out the necessary consent form, and sign off on the risk assessment pertaining to donning the Galea headset specifically. They were then briefed on the headset, and was put on their head, adjusting the headset to ensure a snug fit, that translated into, at minimum, all necessary nodes to be picking up on data.

The participants were sat down in front of the laptop and were asked to focus on a fixation cross for 2 minutes in a quiet, controlled environment to establish a baseline reading for the engagement index at rest. If the reading dips below this it may signify a lack of interest as the participant slips into a bored state. After these 2 minutes had elapsed, the recording was terminated.

The prototype was then opened, and the participants were told to play it. They would interact with the boss and attempt to kill it before they died themselves. Simultaneously, the Galea headset was set to start recording EEG data again. No matter the outcome, whenever someone reached 0 hit points, the game was ended and the recording was stopped for the activity. The headset was then taken off.

Once the activity was concluded, each participant was asked to fill out the Flow Short Scale (FSS) survey. This survey is a subject method of judging one's own flow state. Once the form was filled out, their turn in the study was concluded.

4.6.3 Data Analysis

All data from the headset was parsed into BDF format by a script provided by the University (Buxton, 2026). Using this code, the following steps were followed to calculate the engagement index for both the baseline and the recording during activity, once the data had been recorded:

1. The data from the headset was recorded and saved via Galea GUI into .csv format, within a .txt file.
2. The data was then converted into .bdf format for analysis.
3. The data was split into 30 seconds epochs (Sanei and Chambers, 2013).
4. The average power for each electrode was plot onto a spike graph.
5. The power was plot on a graph per frequency, thus providing each band of waves, allowing to separate theta, alpha and beta bands.
6. The engagement index was calculated at the baseline, for the lower threshold, the engagement index upper threshold for activity, and the average engagement index of each epoch.

The main method for identifying whether the participant is in a flow state using the EEG Engagement Index is to look for sustained activity values within a defined 'flow zone', typically calculated as one or two standard deviations above the individual's resting baseline mean ($\mu+1\sigma$ to $\mu+2\sigma$). Values below this threshold indicate under-stimulation or task disengagement, whereas values exceeding the upper bound strongly suggest cognitive overload, acute frustration, or artifactual interference.

Alongside the quantitative analysis of the EEG readings above, participants were asked to complete a survey using the aforementioned Flow Short Scale (FSS). This scale was formally standardised by Rheinberg, Vollmeyer, and Engeser (2003) in German and later translated into English and other languages. The survey comprises 16 questions. Questions 1-10 relate to the flow state itself, framed by the 6 components of the flow experience (Csikszentmihalyi, 1975), whereas questions 11-13 pertain to a worry component. These 13 questions are answered on a seven-point Likert scale. Questions 14-16 relate to the challenge-skill balance required for the task at hand. Unlike the previous questions, these are measured on a one-to-nine scale.

Using the scores from the questions, a few statistics are calculated and evaluated, according to Rheinberg, Vollmeyer and Engeser (2019): the total flow score, fluency (the sum of questions 2, 4, 5, 7, 8, 9), absorption (the sum of questions 1, 3, 6, 10), the total worry score, and the total and average demand-skill balance score. A high total score indicates that both fluency and absorption are high. However, a medium overall score could be consistent with a high fluency score and a low absorption score, or vice versa. While this may not be the most reliable source of data for identifying a flow state due to its subjective nature, it can still represent the player's perception of their own flow state mindset and can supplement the EEG data evaluation.

4.6.4 Evaluation

During the initial baseline recording, an Academic Associate supervising suggested using a fixation cross. This was originally outside the specification; however, it was implemented for all 3 participants.

One aspect of the gameplay process that was not considered a factor in this investigation was the prospect of the participant dying before the originally planned 10-minute time limit, which would have resulted in a shorter recording

session and consequently less data being collected, although the 2-minute baseline was collected as intended.

During the gameplay session, experience with the specific genre of the prototype, namely third-person action games, affected their readings and overall skill. The opposite seemed to happen with participants who were less experienced.

CHAPTER 5

RESULTS / DISCUSSION

5.1 Introduction

This chapter presents the empirical findings and critical evaluation of the developed DDA prototype. The primary objective of the testing phase was to determine whether a performance-based, Fuzzy Logic adaptive system could mitigate mechanical friction and sustain a player's autotelic flow state. To achieve this, the investigation combined continuous physiological data, captured via an electroencephalogram (EEG) to calculate a real-time Engagement Index (EI), with post-session qualitative self-reporting using the Flow Short Scale (FSS) (Rheinberg, Vollmeyer and Engeser, 2003).

Evaluating the success of this system requires a twofold approach. First, human-computer interaction is analysed by correlating the system's autonomous difficulty adjustments with participants' resulting neurophysiological and subjective responses. Second, the underlying software architecture and development lifecycle must be critically examined. By analysing both the empirical participant data and the mechanical vulnerabilities exposed at runtime, this evaluation measures the prototype's true viability as an accessible, responsive game system.

5.2 Research Results and Statistical Analysis

. As discussed in Chapter 4, after the play session had concluded, each participant completed the Flow Short Scale (FSS) Survey (Rheinberg, Vollmeyer and Engeser, 2003). Key numbers had been identified:

- Total overall flow score
- The fluency score
- The absorption score
- The overall worry score
- The average demands-skill balance rating.

Each participant had a different experience, as shown in the statistics above.

Here are the findings (see Appendix H for full descriptive statistics). Participant 1 reported the most intense flow state in the group (Total Flow = 58), characterised by high levels of automaticity (Fluency = 35) and deep task immersion (Absorption = 22). However, this appeared to be driven more by high pressure than by relaxed engagement. Despite being deeply 'in the zone', Participant 1 reported the highest performance anxiety (Worry = 16) and felt the task was highly demanding relative to their skill level (Balance = 7.67). In conclusion, they were highly engaged yet simultaneously stressed.

Participant 2 experienced a balanced, relaxed flow state (Total Flow = 52). In contrast to Participant 1, they reported very low levels of performance anxiety (Worry = 10) and felt their skills matched the task's difficulty, with a demands-skill balance score nearly dead-centre (Balance = 4.67). Because they were highly comfortable with the task and the challenge, they found task execution exceptionally smooth and effortless (Fluency = 33). However, because they felt the task did not push their upper limits, they were the least captivated by the activity, reporting the least immersion of the group (Absorption = 19). Overall, they had a comfortable, automatic control rather than deep, absorbing captivation.

Finally, Participant 3 reported the lowest overall flow score (Total Flow = 49), though their experience remained highly engaged. Similar to Participant 2, they exhibited very low performance anxiety (Worry = 10) and felt the task demands

were in line with their skill level (Balance = 5.33). Interestingly, their sub-scores revealed a contrasting experience: despite being quite absorbed in the task (Absorption = 21), they struggled to find a smooth rhythm, resulting in the lowest automaticity of the group (Fluency = 28). This suggests that even when they were comfortable and focused, they may have had to consciously exert effort to execute the task rather than relying on second nature.

In addition to the above data, continuous electroencephalographic (EEG) data were recorded for each participant at a baseline and during the activity. This was done to objectively quantify the players' cognitive states and mitigate extraneous variables. Checking for Flow State was not measured by raw Engagement Index (EI) maximums, but instead by mapping the activity data onto individualised thresholds as discussed in Chapter 4.

The need for an individualised threshold was validated by the resting baseline data. The cohort exhibited extreme neurological variance, which affected the initial EI calculations. Participant 1 demonstrated a severely suppressed resting baseline (0.0548) (see Appendix B). This high-Theta, low-Beta profile perfectly mirrors the inverted resting baseline sometimes associated with neurodivergent individuals, such as those with ADHD (Arns et al., 2013). By contrast, Participant 2 exhibited a baseline nearly 6 times higher (EI = 0.3115) (see Appendix D), indicating a neurotypical resting state with greater baseline cortical arousal, despite identifying as neurodivergent in their consent form. Had the analysis relied on a universal average, Participant 1's subsequent data would have been erroneously categorised as severely under-stimulated. Calculating active gameplay strictly as a delta (Δ) relative to individual baselines successfully avoided this conclusion.

During the initial epochs of gameplay, Participant 1 recorded extreme EI spikes, far exceeding their calculated upper threshold (0.1254) (see Appendix C). As

previously defined, such maxima do not correlate with flow and may instead indicate anxiety, hyper-arousal, or artefacts caused by physical tension. The game's mechanical demands initially exceeded the player's processing capacity, breaking immersion. However, as gameplay continued, cortical activity stabilised. By the final epoch, their index (0.1047) fell within their personal Flow Channel plateau. This indicates a successful late-stage transition to optimal management, showing that although flow was not sustained throughout the trial, the participant overcame the initial friction to achieve optimal task absorption.

Throughout the active recording, Participant 2's mean engagement ($EI = 0.2499$) (see Appendix E) fell below the lower bound of their Flow Channel (0.4897). Despite entering the task with high resting arousal, their cortical engagement fell below baseline during gameplay (see Appendix D). Visual analysis of the Power Spectral Density (PSD) data corroborated this drop, showing a distinct suppression of Beta band activity. This indicates a state of boredom and under-stimulation; the mechanical demands of the game were insufficient to engage their resting state.

The data from Participant 3 highlight the risks of unmitigated physical variables. Thresholding initially flagged their first activity epoch ($EI = 2.1384$) (see Appendix G) as within the optimal Flow Channel. However, visual inspection of the PSD graphs revealed major variance in the resting baseline (see Appendix F), driven by severe, irregular artefacts. This artefact-corrupted baseline artificially widened the calculated Flow Channel parameters, effectively categorising physical noise as cognitive engagement. Following this initial artefact, all subsequent gameplay epochs fell well below the baseline mean. Therefore, Participant 3 was not experiencing flow; their true physiological state mirrored the disengagement of Participant 2. The initial "in-zone" reading was a false positive caused by physical interference.

Cross-validating subjective and physiological data revealed different outcomes across participants. Participant 1 moved from acute overload into genuine neurophysiological flow, showing that the DDA successfully reduced their initial mechanical friction despite high self-reported stress. Participant 2 produced a subjective false positive; although they reported effortless play, their EEG indicated under-stimulation because the DDA failed to raise the challenge to match their elevated baseline arousal. Participant 3 showed similar limitations; their early “flow” readings were artifactual, and once corrected, their true data resembled Participant 2’s disengagement. Overall, the DDA guided only Participant 1 into the Flow Channel, demonstrating that effective adaptation requires individualised baselines and artefact-aware processing rather than relying on subjective reports. Furthermore, the DDA must have a greater impact on overall gameplay to potentially have a larger impact on the participants’ numbers.

5.3 Evaluation of Process

5.3.1 Successes and Opportunities

A major success of the development process was the decision to offload some of the complex logic to third-party libraries and frameworks, including Accord.Fuzzy (Souza et al., 2014) framework and Jason Weimann’s Finite State Machine (FSM) foundational logic (Weimann, 2020). This greatly streamlined the programming phase and improved the legibility of the codebase.

In addition, the implementation of Git version control proved invaluable. During a migration from Unity 6.2 to 6.3, the project experienced a severe regression. The ability to roll back to a stable version saved the prototype from catastrophic failure, highlighting the need for systematic event logs in future software projects.

Lastly, calculating the Engagement Index (EI) using an individualised delta (Δ) baseline was a profound success, as it prevented Participant 1's neurodivergent baseline from skewing the active telemetry.

5.3.2 Limitations and Challenges

The primary challenge during development was managing the project timeline alongside overlapping academic deadlines, forcing a compressed Agile-Scrum approach that left some bugs unresolved.

During the testing phase, the methodology aimed to collect 10 minutes of continuous gameplay. However, because participants either died or beat the enemy before the time limit was reached, the datasets were prematurely truncated. While effective for acute biometric spikes, this lacked the longitudinal volume required for long-term moving averages.

Additionally, data analysis revealed the vulnerability of EEG to physical artifacts, as Participant 3's initial "flow" readings were possibly false positives driven by physical movement noise corrupting the baseline.

5.4 Evaluation of the Prototype

During the recording session, when the participants were playing the prototype, the state machine was configured so that whenever the enemy took damage, their current state was interrupted. In tandem with the short cooldown between taking damage, this allowed the player to "spam" attack inputs, effectively locking the enemy in one spot and preventing the enemy from functioning as intended.

Version control was used, albeit not from the start, which helped keep the project in a stable state. For example, the prototype encountered a major issue when updating from Unity 6.2 to 6.3, resulting in the game breaking entirely. If

it were not for the prior version stored in Git and Unity version control, the prototype would have had to be rebuilt from the ground up.

- Researching for third-party libraries and scripts streamlined the process immensely. For example, using Accord.Fuzzy as a library to cover the complex mathematics required to program specific functions such as Centroid Defuzzification. This improved the overall quality of the code and more legible.

CHAPTER 6

CONCLUSIONS / FUTURE WORK

6.1 Conclusions

The primary objective of this investigation was to determine whether an autonomous, performance-based Dynamic Difficulty Adjustment (DDA) system could effectively sustain a player's flow state, thereby supporting those with potential psychomotor or processing barriers. By combining the flow-theory framework with practical software engineering, the project met its core aims and highlighted the limitations of relying solely on performance-based adaptation.

In relation to Aim 1, the project performed strongly. The prototype showed that an Accord.Fuzzy inference system could be integrated smoothly into a Unity-based finite state machine. By tracking reaction speed and dodge/block success in real time, the system converted raw numbers into linguistic categories and adjusted the enemy's speed and damage accordingly. This demonstrated that performance-based DDA can operate invisibly within an action-focused gameplay loop, maintaining immersion without requiring the player to interact with any difficulty settings.

Aim 2 yielded mixed but still valuable findings. The EEG data confirmed that subjective self-reporting is not a reliable indicator of flow. For Participant 1, the Engagement Index clearly showed that the adaptive system worked as intended: it reduced an initial mechanical bottleneck and helped guide the player from a hyper-aroused state to a stable flow plateau.

Participants 2 and 3, however, exposed the limits of relying solely on performance metrics. Participant 2 reported feeling comfortable, yet their EEG data showed clear under-stimulation, indicating the system failed to increase the challenge sufficiently to match their naturally high baseline. Participant 3's corrupted baseline also highlighted how sensitive EEG hardware is to movement artefacts during active play, and how easily physiological data can be misinterpreted without proper filtering.

Overall, the project shows that while performance-based fuzzy logic is an effective and accessible way to balance mechanical difficulty, it cannot reliably infer a player's internal cognitive state on its own. Future adaptive systems will need to combine performance data with robust biometric measures, provided that hardware continues to improve at filtering out movement-related noise.

6.2 Future work

While the prototype successfully established a proof-of-concept for biometric validation of a performance-based DDA, several avenues for future development have been identified to mature the system from an investigative foundation into a more viable product or study:

- The most immediate technical priority is to resolve the state machine vulnerabilities identified during testing. Future iterations must implement a robust 'poise' or invulnerability mechanic within the boss's damage-handling logic. This would alleviate the 'stunlock' exploit, ensuring the telemetry data fed to the Fuzzy Logic controller genuinely reflects the player's psychomotor skill rather than their ability to bypass the combat loop.
- Currently, the DDA modifies floating-point values (animation speed and damage multipliers). To exert a more profound impact on the player's

cognitive state, future versions of the Fuzzy Logic matrix should be expanded to influence the enemy's decision-making tree directly. For example, locking specific attack patterns based on the Engagement Index or player performance, rather than just altering the speed or damage of attacks.

- At present, the prototype functions only as a performance-based DDA, utilising EEG data exclusively for post-hoc validation. The next step for this architecture is to evolve it into a true multi-measure, hybrid system (Fisher and Kulshreshth, 2024). By engineering the Accord.Fuzzy controller to ingest real-time cognitive information, specifically by streaming the participant's live Engagement Index (EI) alongside mechanical telemetry, the system could execute emotion-based adaptations. This would enable the AI to pre-emptively de-escalate mechanical friction the moment hyper-arousal (sensory overload) is detected, rather than waiting for the player's psychomotor performance to degrade. Implementing a real-time feedback loop would fully realise the accessible adaptation models advocated in current serious games literature.
- The current methodology relied heavily on quantitative EEG data, supported by a brief Likert-scale survey. Future studies may integrate more comprehensive qualitative data collection, such as post-session thematic interviews. This mixed-methods approach would provide additional context, clarifying exactly why a neurodivergent participant's Engagement spiked at a specific moment.
- Not enough data was collected throughout the investigation. Having fewer than 10 minutes of records, on top of a smaller cohort, is insufficient for establishing stable, long-term EEG thresholds and for adopting a more holistic approach by incorporating greater variance in the data. Future investigations should utilise extended recording sessions

across multiple encounters. This allows the dynamic standard deviation thresholds to refine themselves over time, providing a much higher resolution of the player's sustained autotelic state.

- When vetting and looking for participants, attempting to find people of the target group, in this case people with psychomotor impairments or similar.

6.3 Legal, Social, Ethical and Professional Issues

6.3.1 Legal Issues

- Using a standalone prototype rather than modifying an existing commercial game avoided any intellectual property complications. This ensured that all testing, analysis, and written material remained legally compliant and free from copyright concerns.
- Participant recruitment followed the Equality Act 2010 to ensure everyone was treated fairly and without discrimination. This protected the integrity of the study by guaranteeing that all volunteers were selected and supported in accordance with the same legal and ethical standards throughout the entire testing process.
- Because continuous EEG recordings constitute "Special Category Data" under the GDPR, strict data-protection measures were required. All biometric information was anonymised, securely stored, and handled in full legal compliance to safeguard each participant's privacy and protect their sensitive neurological data from potential misuse.

6.3.2 Social Issues

- Participants needed to be fluent in English to fully understand the study, surveys, and gameplay instructions. While necessary for clarity and

safety, this requirement inevitably narrowed the demographic range of the final sample and limited the data's broader social representativeness.

- Creating a respectful and inclusive testing environment was essential. By actively monitoring for unconscious bias and treating all feedback impartially, the study maintained strong objectivity, enabling its conclusions to genuinely support accessible, inclusive game design for a wide range of players and abilities.
- The need for direct-to-scalp EEG sensors unintentionally excluded individuals who wear religious head coverings, such as hijabs and turbans. Because removing these garments cannot be ethically requested, future biometric accessibility research must address this hardware limitation to ensure more representative and inclusive participation.

6.3.3 Ethical Issues

- Transparent, informed consent was prioritised from the outset. All potential triggers and requirements were clearly explained so participants fully understood the study, ensuring that their involvement was voluntary and that the resulting feedback and data were ethically collected and scientifically reliable.
- Testing the adaptive system carried a risk of short-term frustration. Continuous EEG monitoring served as an ethical safeguard, reinforcing that modern accessibility research must consider a player's mental well-being and emotional safety alongside their physical comfort and mechanical performance.

6.3.4 Professional Issues

- All physiological data was analysed exactly as recorded, without alteration or selective filtering. Maintaining strict data integrity was essential to make sure that the final results remained accurate, trustworthy, and professionally credible throughout the evaluation and interpretation stages of the research.
- Professional conduct demands complete transparency throughout the project. This report transparently details all methodological challenges, unforeseen issues, and hardware limitations faced, providing the academic community with an honest, thorough, and dependable overview of the project's strengths and weaknesses.

6.4 Synoptic Reflections

Undertaking this project has put my programming abilities to the test, particularly in Object-Oriented Programming (OOP). Since the first year, this has been a recurring theme, with regular practice, particularly within the field of game technology. Furthermore, the more advanced aspects of Unity Engine have been really insightful, revealing the engine's utility and application potential outside of a gaming context.

This project has been invaluable in teaching me the importance of both keeping a log and of project management. Whether it is a development log, progress tracking, or version control, keeping notes is a skill I will carry beyond undergraduate level, into postgraduate and beyond. If there is anything relating to project management this experience has taught me, it is that "time is money"; spread the work out over the whole allotted period and avoid last-minute sprints.

Working with Extended Reality, Human-Computer Interaction aligns with the postgraduate route I aspire to take and with future employment. The aim is to

work in Research and Development, whether in an entertainment context or another field.

REFERENCES

- Arns, M., Conners, C.K. and Kraemer, H.C. (2013) 'A Decade of EEG Theta/Beta Ratio Research in ADHD: A Meta-Analysis', *Journal of Attention Disorders*, 17(5), pp. 374–383. Available at: <https://doi.org/10.1177/1087054712460087>.
- Aytemiz, B. et al. (2020) 'Your Buddy, the Grandmaster: Repurposing the Game-Playing AI Surplus for Inclusivity', *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 16(1), pp. 17–23. Available at: <https://doi.org/10.1609/aiide.v16i1.7402>.
- Buxton, Dan. (2026) *GaleaTools* [Repository]. Available at: <https://olympus.ntu.ac.uk/cmp3buxtod/GaleaTools> (Accessed: 6 May 2026)
- Capcom. (2005). *Resident Evil 4* [Video game]. Capcom. Nintendo GameCube.
- Carvalho, C. et al. (2024) 'Technologies to Support Adaptable Game Design: A Systematic Mapping Study', *Journal of the Brazilian Computer Society*, 30(1), pp. 69–101. Available at: <https://doi.org/10.5753/jbcs.2024.3090>.
- Csikszentmihalyi, M. (1975) *Beyond boredom and anxiety*. 1st ed. San Francisco: Jossey-Bass Publishers (The Jossey-Bass behavioral science series).
- Csikszentmihalyi, M. (2008) *Flow: The psychology of optimal experience*. Nachdr. New York: Harper [and] Row (Harper Perennial Modern Classics).
- Data Protection Act 2018, c.12. Available at: <https://www.legislation.gov.uk/ukpga/2018/12/contents> (Accessed: 6 May 2026)
- Dickson, T. & S. Dolnicar (2004). *No risk, no fun: The role of perceived risk in adventure tourism*. [online] Available at:

https://www.researchgate.net/publication/30389108_No_risk_no_fun_The_role_of_perceived_risk_in_adventure_tourism.

Fairclough, S.H. (2009) 'Fundamentals of physiological computing', *Interacting with Computers*, 21(1–2), pp. 133–145. Available at: <https://doi.org/10.1016/j.intcom.2008.10.011>.

Fisher, N. and Kulshreshth, A.K. (2024) 'Exploring Dynamic Difficulty Adjustment Methods for Video Games', *Virtual Worlds*, 3(2), pp. 230–255. Available at: <https://doi.org/10.3390/virtualworlds3020012>. (Accessed: 20 April 2026)

Fisher, N. and K. Kulshreshth, A. (2025) 'Dynamic Difficulty Adjustment in Games: Concepts, Techniques, and Applications', in T. Guzsvinecz (ed.) *From Pixels to Play - The Art and Science of Video Games*. IntechOpen. Available at: <https://doi.org/10.5772/intechopen.1011703>.

Gillin, J.L. and Huizinga, J. (1951) 'Homo Ludens: A Study of the Play-Element in Culture.', *American Sociological Review*, 16(2), p. 274. Available at: <https://doi.org/10.2307/2087716>.

Hendrix, M. et al. (2019) 'Implementing Adaptive Game Difficulty Balancing in Serious Games', *IEEE Transactions on Games*, 11(4), pp. 320–327. Available at: <https://doi.org/10.1109/TG.2018.2791019>.

Hunicke, R. (2005) 'The case for dynamic difficulty adjustment in games', *Proceedings of the 2005 ACM SIGCHI International Conference on Advances in computer entertainment technology. ACE05: International Conference on Advances in Computer Entertainment Technology*, Valencia Spain: ACM, pp. 429–433. Available at: <https://doi.org/10.1145/1178477.1178573>.

Katahira, K. et al. (2018) 'EEG Correlates of the Flow State: A Combination of Increased Frontal Theta and Moderate Frontocentral Alpha Rhythm in the Mental

Arithmetic Task', *Frontiers in Psychology*, 9, p. 300. Available at:

<https://doi.org/10.3389/fpsyg.2018.00300>.

Kirschstein, T. and Köhling, R. (2009) 'What is the Source of the EEG?', *Clinical EEG and Neuroscience*, 40(3), pp. 146–149. Available at:

<https://doi.org/10.1177/155005940904000305>.

Maxylobes (2024). *The Controversial Mechanic That Changed Resident Evil*.

[online] YouTube. Available at:

<https://www.youtube.com/watch?v=pR00u3ltyuY> [Accessed 3 May 2026]

McMahan, T., Parberry, I. and Parsons, T.D. (2015) 'Evaluating Player Task Engagement and Arousal Using Electroencephalography', *Procedia Manufacturing*, 3, pp. 2303–2310. Available at:

<https://doi.org/10.1016/j.promfg.2015.07.376>.

Maddy Makes Games Inc. (2018). *Celeste* [Video game]. Maddy Makes Games Inc. PC.

Monastra, V.J., Lubar, J.F., Linden, M., VanDeusen, P., Green, G., Wing, W., Phillips, A. and Fenger, T.N. (1999) 'Assessing attention deficit hyperactivity disorder via quantitative electroencephalography: an initial validation study', *Neuropsychology*, 13(3), pp. 424–433. Available at:

<https://doi.org/10.1037/0894-4105.13.3.424>

Nacke, L. and Lindley, C.A. (2008) 'Flow and Immersion in First-Person Shooters: Measuring the player's gameplay experience'. Available at:

<https://doi.org/10.1145/1496984.1496998>

OpenBCI. (2024). *Galea Headset* (Beta version) [Equipment]. Brooklyn, NY: OpenBCI.

Rheinberg, F., Vollmeyer, R. and Engeser, S. (2003) 'Die Erfassung des Flow-Erlebens', in J. Stiensmeier-Pelster and F. Rheinberg (eds) *Diagnostik von Motivation und Selbstkonzept*. Göttingen: Hogrefe, pp. 261-279.

Seyderhelm, A.J.A. and Blackmore, K. (2021) 'Systematic Review of Dynamic Difficulty Adaption for Serious Games: The Importance of Diverse Approaches', *SSRN Electronic Journal* [Preprint]. Available at: <https://doi.org/10.2139/ssrn.3982971>.

Salen, K. and Zimmerman, E. (2004) *Rules of play: game design fundamentals*. Cambridge, Mass: MIT Press.

Sanei, S., and Chambers, J. A. (2013) *EEG Signal Processing*. 2nd edn. Oxford: Wiley-Blackwell.

Schwaber, K. and Sutherland, J. (2020) *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. [Online]. Available at: <https://scrumguides.org/scrum-guide.html> (Accessed: 8 May 2026)

Souza, C., Kirilov, A., Catalano, M. D. and Accord.NET contributors (2014) *The Accord.NET Framework* [Software]. Available at: <http://accord-framework.net/> (Accessed: 7 May 2026).

Unity Technologies. (2024). *Start Assets – Third Person Character Controller* (Version 1.1.7) [Software]. Available at: <https://assetstore.unity.com/packages/essentials/starter-assets-thirdperson-urp-196526> (Accessed: 8 May 2026)

Weimann, J. (2020) *Unity Bots with State Machines – Extensible State Machine / FSM*. [Online video]. YouTube. Available at: <https://www.youtube.com/watch?v=V75hgcsCGOM> (Accessed: 8 May 2026)

Zohaib, M. (2018) 'Dynamic Difficulty Adjustment (DDA) in Computer Games: A Review', *Advances in Human-Computer Interaction*, 2018, pp. 1–12. Available at: <https://doi.org/10.1155/2018/5681652>.

APPENDIX A

DdaInputTracker.cs:

```
using System.Collections.Generic;
using UnityEngine;
using System.Linq;

public class DdaInputTracker : MonoBehaviour
{
    [Header("References")] public FuzzyBrain ddaEngine;

    [Header("Tracking Data")]
    private Queue<float> _reactionHistory = new Queue<float>();
    private Queue<bool> _hitHistory = new Queue<bool>();
    private float _attackStartTime;

    private bool _isReactionTracking = false;

    private float _ddaTimer = 0f;
    private const float ddaCheckInterval = 35f;

    void Update()
    {
        _ddaTimer += Time.deltaTime;
        if (_ddaTimer >= ddaCheckInterval)
        {
            _ddaTimer = 0f;
            UpdateDDA();
        }
    }

    public void RecordAttackStart()
    {
        _attackStartTime = Time.time;
        _isReactionTracking = true;
    }

    public void RecordPlayerResponse()
    {
        if (_isReactionTracking)
        {
            if (_reactionHistory.Count > 5)
            {
                _reactionHistory.Dequeue();
            }

            float reactionTime = Time.time - _attackStartTime;

            if (reactionTime > 2f)
            {
                Debug.Log("Reaction Time was too high for DDA: " +
                    reactionTime + ". Setting the reaction speed to maximum 2.0s.");
            }
        }
    }
}
```

```

        reactionTime = Mathf.Clamp(reactionTime, 0.01f, 2.0f);

        _reactionHistory.Enqueue(reactionTime);

        _isReactionTracking = false;

        Debug.Log($"Player reacted in: {reactionTime:F2}");
    }
}

public void RecordOutcome(bool playerWasHit)
{
    _hitHistory.Enqueue(playerWasHit);

    if (_hitHistory.Count > 10)
    {
        _hitHistory.Dequeue();
    }

    float successCount = 0;
    foreach (bool wasHit in _hitHistory)
    {
        if (!wasHit) successCount++;
    }

    float successRate = (successCount / _hitHistory.Count) *
100f;

    ddaEngine.currentPlayerSuccessRate =
Mathf.Clamp(successRate, 0f, 100f);
}

public void UpdateDDA()
{
    float totalReactionTime = 0f;
    for (int i = 0; i < _reactionHistory.Count; i++)
    {
        totalReactionTime += _reactionHistory.ElementAt(i);
    }

    float averageReactionTime = totalReactionTime /
_reactionHistory.Count;

    ddaEngine.currentPlayerReactionSpeed =
Mathf.Clamp(averageReactionTime, 0.01f, 2.0f);

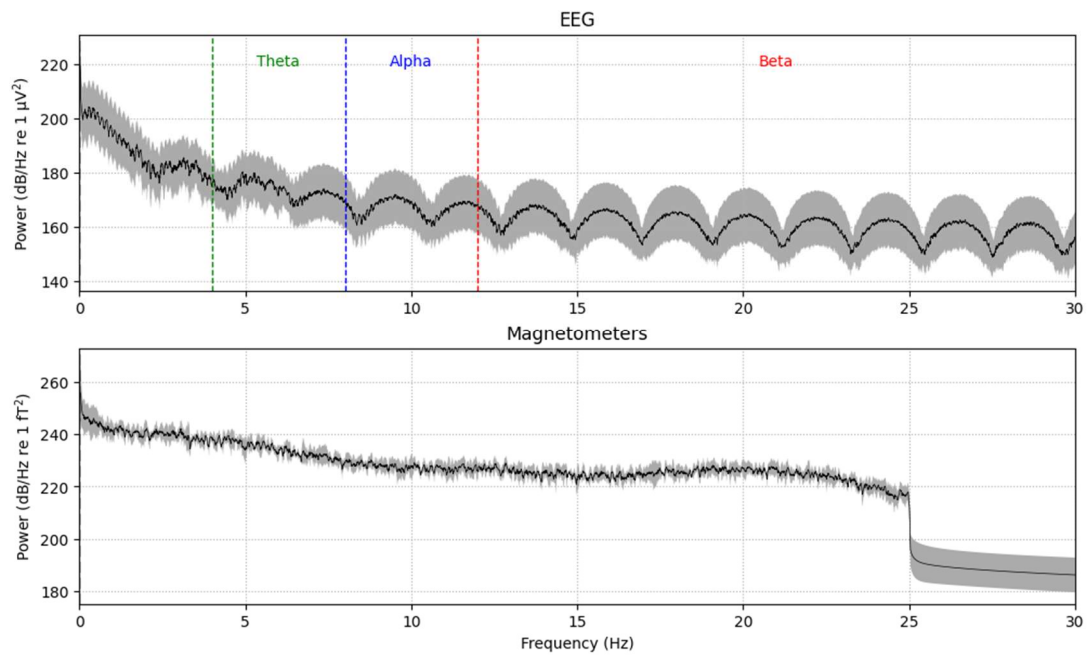
    Debug.Log("Total values in reaction history: " +
_reactionHistory.Count);
    Debug.Log("Average reaction time: " + averageReactionTime);

    ddaEngine.EvaluatePlayer(ddaEngine.currentPlayerReactionSpeed,
ddaEngine.currentPlayerSuccessRate);
}
}

```

APPENDIX B

Participant 1 EEG reading baseline:



#1 baseline

Engagement: [0.01774483 0.02211675 0.08256892 0.09682593]

Mean: 0.0548

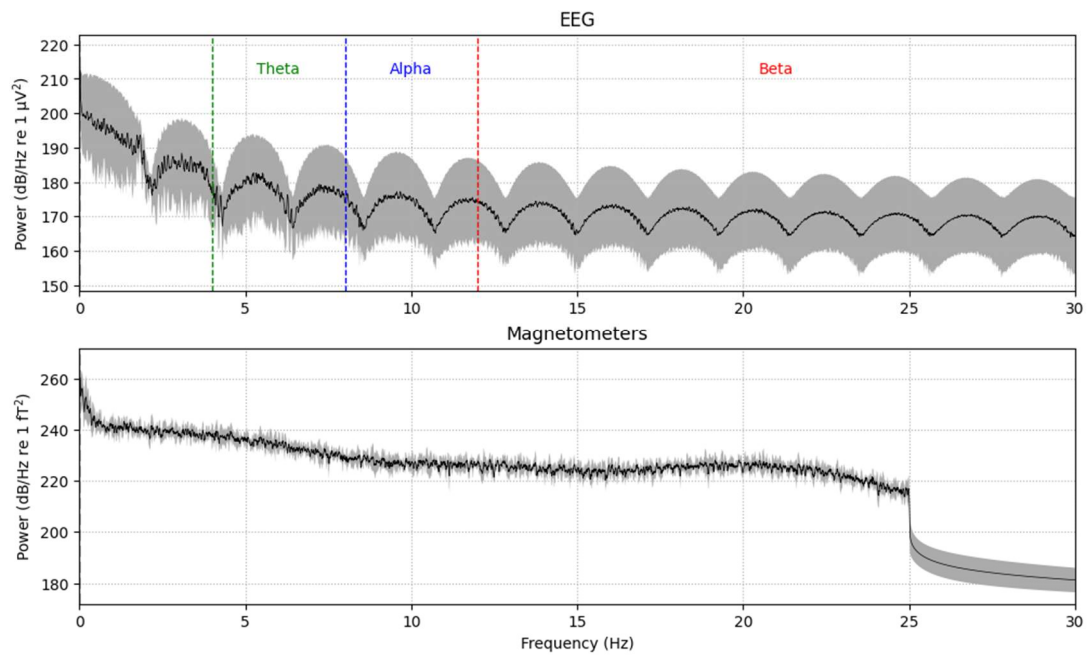
SD: 0.0353

Lower: 0.0195

Upper: 0.0901

APPENDIX C

Participant 1 EEG reading activity:



#1 activity

Engagement: [2.24475995 0.21729907 0.89094197 0.32873871 0.10469494]

Mean: 0.7573

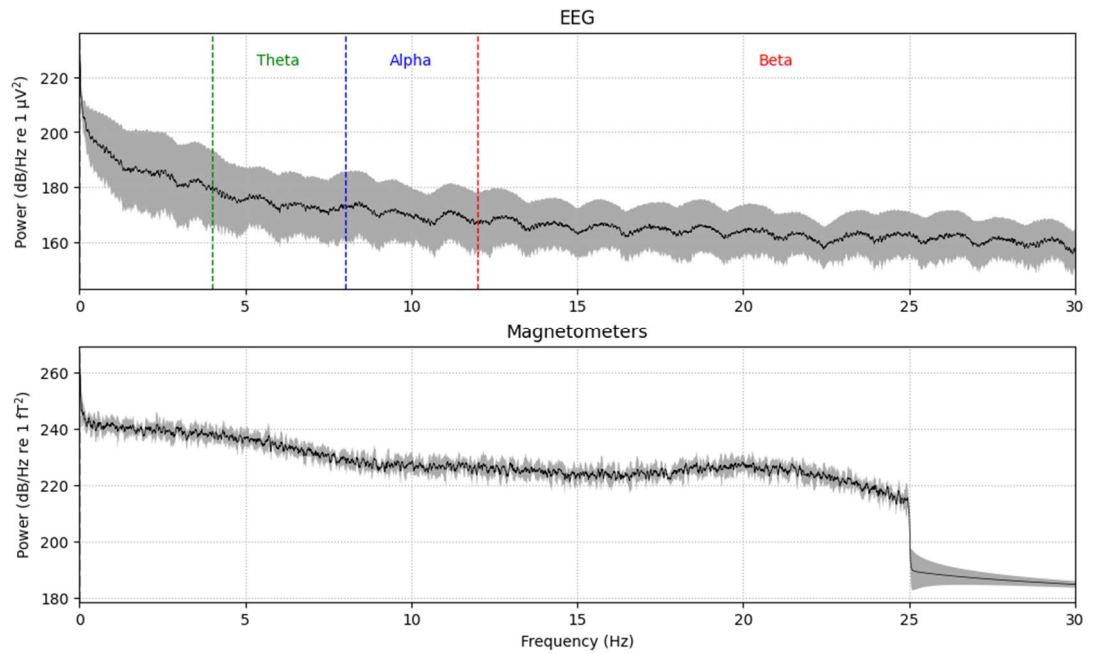
SD: 0.7914

Lower: -0.0341

Upper: 1.5487

APPENDIX D

Participant 2 EEG reading baseline:



#2 baseline

Engagement: [0.58557727 0.34139531 0.11085491 0.20835536]

Mean: 0.3115

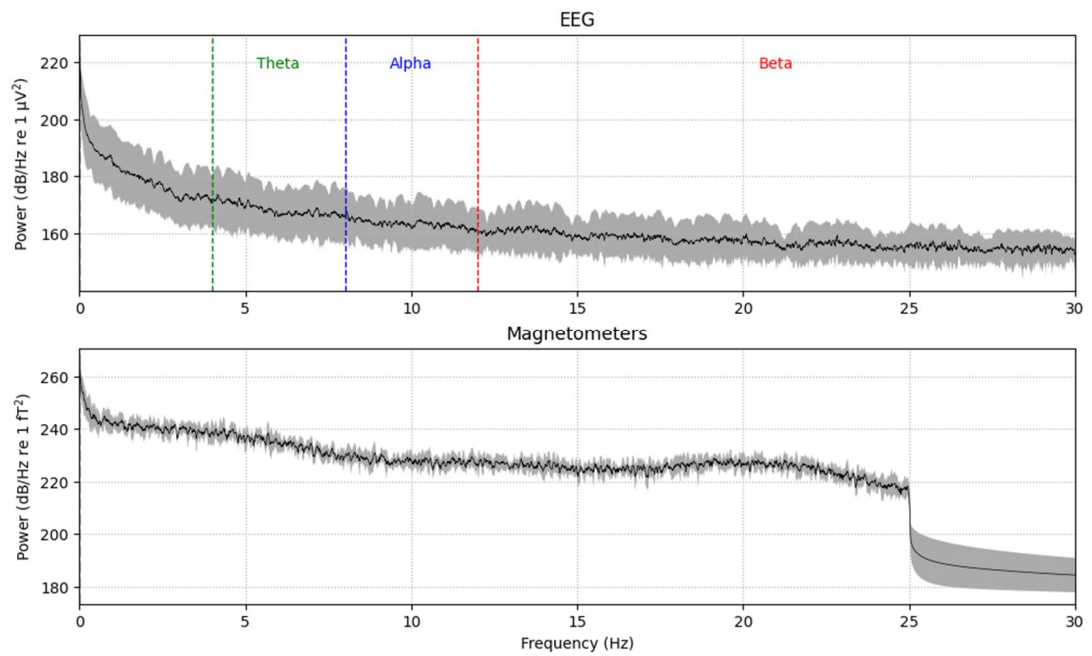
SD: 0.1781

Lower: 0.1334

Upper: 0.4897

APPENDIX E

Participant 2 EEG reading activity:



#2 activity

Engagement: [0.27378358 0.20230677 0.27348073]

Mean: 0.2499

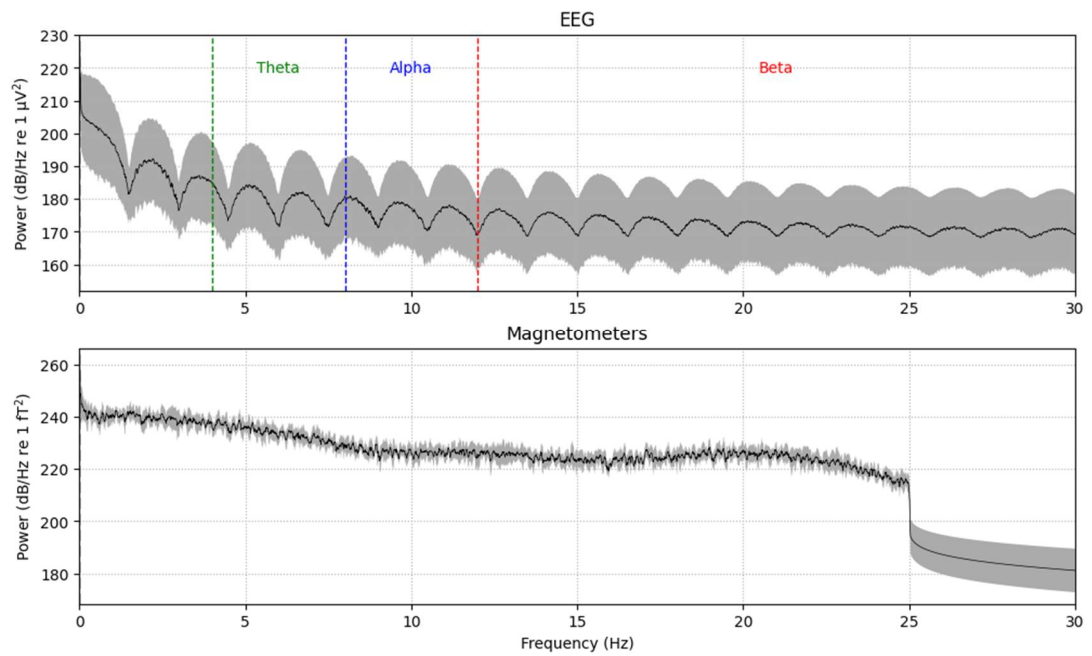
SD: 0.0336

Lower: 0.2162

Upper: 0.2835

APPENDIX F

Participant 3 EEG reading baseline:



#3 baseline

Engagement: [2.11410923 0.22695005 0.17141846 0.0869074]

Mean: 0.6498

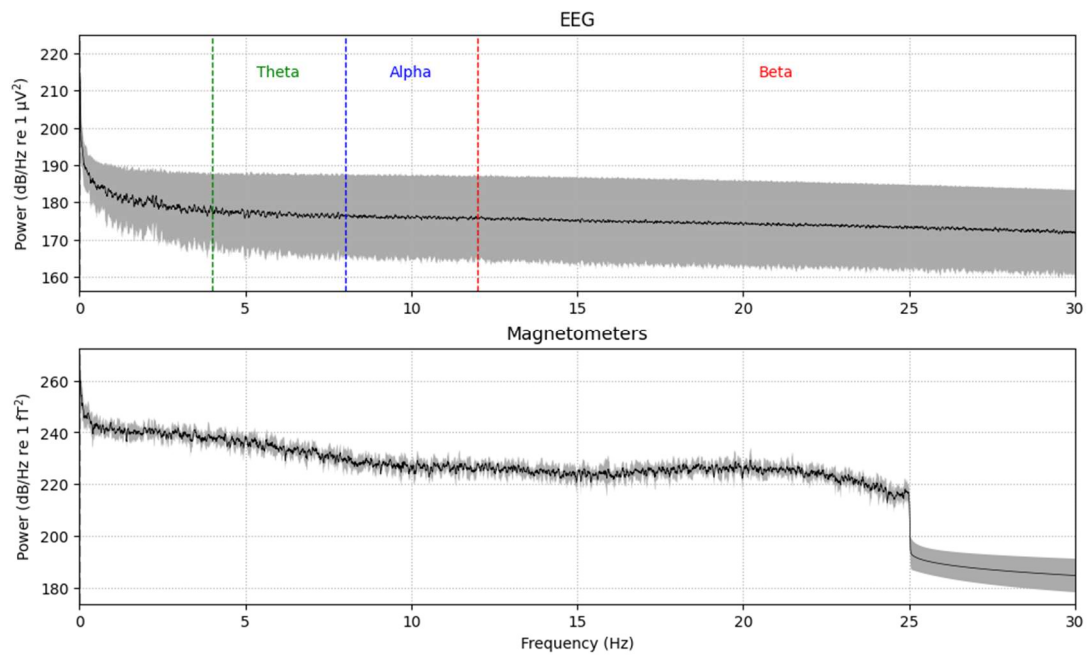
SD: 0.8469

Lower: -0.1970

Upper: 1.4967

APPENDIX G

Participant 3 EEG reading activity:



#3 activity

Engagement: [2.13841425 0.35402708 0.50025941 0.46995125]

Mean: 0.8657

SD: 0.7368

Lower: 0.1288

Upper: 1.6025

APPENDIX H

Flow Short Scale Result Table:

Flow Experience (1-7)												
Participant No.	Question No.	1	2	3	4	5	6	7	8	9	10	
	1	5	6	3	7	6	7	5	6	6	7	
	2	3	6	7	6	6	7	4	6	5	2	
	3	6	6	6	6	4	5	5	4	3	4	
Performance Concerns (1-7)				Demands-Skill Balance (1-9)								
Participant No.	Question No.	11	12	13	14	15	16					
	1	5	5	6	7	8	8					
	2	3	2	5	2	7	5					
	3	4	4	2	6	5	5					
Participant No.	Flow Score Total	Fluency Score	Absorption Score	Worry Score Total	Total Balance Score	Average Rating						
1	58	35	22	16	23	7.67						
2	52	33	19	10	14	4.67						
3	49	28	21	10	16	5.33						

APPENDIX I

EnemyStateMachine.cs:

```
using System;
using UnityEngine;
using UnityEngine.AI;

public class EnemyStateMachine : MonoBehaviour
{
    public Transform player;

    public float detectionRadius = 15f;
    public float attackRange = 2.5f;

    public DamageDealer axe;
    public DamageDealer foot;

    public float baseMovementSpeed = 3.5f;

    private StateMachine _stateMachine;
    private Animator _animator;
    private NavMeshAgent _agent;
    private Health _health;

    public float currentDdaSpeed = 1f;
    public float currentDamageMod;

    [HideInInspector] public bool isAttacking = false;
    [HideInInspector] public bool isDead = false;

    public float phase2DamageMultiplier = 1.15f;
    [HideInInspector] public bool isPhase2 = false;

    private void Awake()
    {
        player = GameObject.FindGameObjectWithTag("Player").transform;

        _agent = GetComponent<NavMeshAgent>();
        _agent.updateRotation = false;

        _animator = GetComponent<Animator>();
        var playerDetector = GetComponentInChildren<PlayerDetector>();

        _health = GetComponent<Health>();

        _stateMachine = new StateMachine();

        var patrol = new PatrolState(this, _agent);
        var chase = new ChasePlayerState(this, _agent, player);
        var attack = new AttackState(this, _agent, player, _animator, axe, foot);
        var death = new DeathState(_animator, _agent);

        var phase2Transition = new PhaseTransitionState(this, _animator, _agent, _health);

        At(patrol, chase, () => DistanceToPlayer() <= detectionRadius);
        At(chase, patrol, () => DistanceToPlayer() > detectionRadius);

        At(chase, attack, () => DistanceToPlayer() <= attackRange);

        At(attack, chase, () => !isAttacking);
        At(attack, patrol, () => !isAttacking && !playerDetector.PlayerInRange);

        _stateMachine.AddAnyTransition(death, () => isPhase2 && isDead);
        _stateMachine.AddAnyTransition(phase2Transition, () => _health != null && _health.currentHealth > 0 &&
        _health.currentHealth <= (_health.maxHealth * 0.5f) && !isPhase2 && !isDead);

        At(phase2Transition, chase, () => phase2Transition.IsComplete);

        _stateMachine.SetState(patrol);

        void At(IState from, IState to, Func<bool> condition) => _stateMachine.AddTransition(from, to,
        condition);
    }

    void Update()
    {
        _stateMachine.Tick();

        if (_animator != null) _animator.speed = currentDdaSpeed;

        if (_agent != null) _agent.speed = baseMovementSpeed * currentDdaSpeed;

        UpdateBlendTree();
    }
}
```

```

        FacePlayer();
    }

    public void FacePlayer()
    {
        if (player == null) return;

        Vector3 direction = (player.position - transform.position).normalized;
        direction.y = 0;

        if (direction != Vector3.zero)
        {
            Quaternion baseRotation = Quaternion.LookRotation(direction);

            Quaternion correctedRotation = baseRotation * Quaternion.Euler(0, 45f, 0);

            transform.rotation = Quaternion.Slerp(transform.rotation, correctedRotation, Time.deltaTime * 8f);
        }
    }

    private void OnAnimatorMove()
    {
        if (isAttacking && _animator != null)
        {
            transform.position += _animator.deltaPosition;
            transform.rotation *= _animator.deltaRotation;
        }
    }

    private void UpdateBlendTree()
    {
        if (_animator != null && _agent != null)
        {
            float currentSpeed = _agent.velocity.magnitude;
            currentSpeed = Mathf.Clamp(currentSpeed, 0f, 5f);

            _animator.SetFloat("Speed", currentSpeed);
        }
    }

    private float DistanceToPlayer()
    {
        if (player == null) return float.MaxValue;

        Vector3 playerPos = new Vector3(player.position.x, 0, player.position.z);
        Vector3 bossPos = new Vector3(transform.position.x, 0, transform.position.z);

        return Vector3.Distance(playerPos, bossPos);
    }
}

```

APPENDIX J

FuzzyBrain.cs:

```
using UnityEngine;
using Accord.Fuzzy;

public class FuzzyBrain : MonoBehaviour
{
    private InferenceSystem engine;
    private EnemyStateMachine _stateMachine;

    [Header("Player Tracking")]
    public float currentPlayerReactionSpeed = 1.0f;
    public float currentPlayerSuccessRate = 50f;

    [Header("Base Boss Stats")]
    public float baseEnemySpeed = 1f;
    public float baseEnemyDamage;

    [HideInInspector] public float calculatedAnimSpeed;
    [HideInInspector] public float calculatedDamageMod;

    void Awake()
    {
        BuildBrain();

        _stateMachine = GetComponent<EnemyStateMachine>();

        baseEnemyDamage = _stateMachine.axe.damageAmount;
    }

    private void BuildBrain()
    {
        // Reaction speed input
        LinguisticVariable reactionSpeed = new LinguisticVariable("ReactionSpeed", 0f, 2.0f);
        reactionSpeed.AddLabel(new FuzzySet("Fast", new TrapezoidalFunction(0f, 0f, 0.3f, 0.6f)));
        reactionSpeed.AddLabel(new FuzzySet("Average", new TrapezoidalFunction(0.4f, 0.6f, 1.0f, 1.3f)));
        reactionSpeed.AddLabel(new FuzzySet("Slow", new TrapezoidalFunction(1.1f, 1.5f, 2.0f, 2.0f)));

        // Success rate input
        LinguisticVariable successRate = new LinguisticVariable("SuccessRate", 0f, 100f);
        successRate.AddLabel(new FuzzySet("Low", new TrapezoidalFunction(0f, 0f, 30f, 45f)));
        successRate.AddLabel(new FuzzySet("Medium", new TrapezoidalFunction(35f, 45f, 65f, 75f)));
        successRate.AddLabel(new FuzzySet("High", new TrapezoidalFunction(65f, 80f, 100f, 100f)));

        // Animation speed output
        LinguisticVariable animSpeed = new LinguisticVariable("AnimationSpeed", 0.7f, 1.2f);
        animSpeed.AddLabel(new FuzzySet("Slow", new TrapezoidalFunction(0.7f, 0.7f, 0.8f, 0.9f)));
        animSpeed.AddLabel(new FuzzySet("Normal", new TrapezoidalFunction(0.85f, 0.95f, 1.05f, 1.1f)));
        animSpeed.AddLabel(new FuzzySet("Fast", new TrapezoidalFunction(1.05f, 1.15f, 1.2f, 1.2f)));

        // Damage multiplier output
        LinguisticVariable dmgMultiplier = new LinguisticVariable("DamageMultiplier", 0.5f, 1.5f);
        dmgMultiplier.AddLabel(new FuzzySet("Low", new TrapezoidalFunction(0.5f, 0.5f, 0.7f, 0.9f)));
        dmgMultiplier.AddLabel(new FuzzySet("Normal", new TrapezoidalFunction(0.8f, 0.9f, 1.1f, 1.2f)));
        dmgMultiplier.AddLabel(new FuzzySet("High", new TrapezoidalFunction(1.1f, 1.3f, 1.5f, 1.5f)));

        Database db = new Database();
        db.AddVariable(reactionSpeed);
        db.AddVariable(successRate);
        db.AddVariable(animSpeed);
        db.AddVariable(dmgMultiplier);

        engine = new InferenceSystem(db, new CentroidDefuzzifier(1000));

        // Rules for fast reaction speed
        engine.NewRule("R_FastButLow_AnimSpeed",
            "IF ReactionSpeed IS Fast AND SuccessRate IS Low THEN AnimationSpeed IS Normal");
        engine.NewRule("R_FastButLow_DmgMultiplier",
            "IF ReactionSpeed IS Fast AND SuccessRate IS Low THEN DamageMultiplier IS Low");

        engine.NewRule("R_FastButMedium_AnimSpeed",
            "IF ReactionSpeed IS Fast AND SuccessRate IS Medium THEN AnimationSpeed IS Fast");
        engine.NewRule("R_FastButMedium_DmgMultiplier",
            "IF ReactionSpeed IS Fast AND SuccessRate IS Medium THEN DamageMultiplier IS Normal");

        engine.NewRule("R_FastButHigh_AnimSpeed",
            "IF ReactionSpeed IS Fast AND SuccessRate IS High THEN AnimationSpeed IS Fast");
        engine.NewRule("R_FastButHigh_DmgMultiplier",
            "IF ReactionSpeed IS Fast AND SuccessRate IS High THEN DamageMultiplier IS High");

        // Rules for average reaction speed
        engine.NewRule("R_AverageButLow_AnimSpeed",
            "IF ReactionSpeed IS Average AND SuccessRate IS Low THEN AnimationSpeed IS Normal");
        engine.NewRule("R_AverageButLow_DmgMultiplier",
            "IF ReactionSpeed IS Average AND SuccessRate IS Low THEN DamageMultiplier IS Low");

        engine.NewRule("R_AverageButMedium_AnimSpeed",
            "IF ReactionSpeed IS Average AND SuccessRate IS Medium THEN AnimationSpeed IS Normal");
        engine.NewRule("R_AverageButMedium_DmgMultiplier",
            "IF ReactionSpeed IS Average AND SuccessRate IS Medium THEN DamageMultiplier IS Normal");
    }
}
```

```

engine.NewRule("R_AverageButHigh_AnimSpeed",
    "IF ReactionSpeed IS Average AND SuccessRate IS High THEN AnimationSpeed IS Normal");
engine.NewRule("R_AverageButHigh_DmgMultiplier",
    "IF ReactionSpeed IS Average AND SuccessRate IS High THEN DamageMultiplier IS High");

// Rules for slow reaction speed
engine.NewRule("R_SlowButLow_AnimSpeed",
    "IF ReactionSpeed IS Slow AND SuccessRate IS Low THEN AnimationSpeed IS Slow");
engine.NewRule("R_SlowButLow_DmgMultiplier",
    "IF ReactionSpeed IS Slow AND SuccessRate IS Low THEN DamageMultiplier IS Low");

engine.NewRule("R_SlowButMedium_AnimSpeed",
    "IF ReactionSpeed IS Slow AND SuccessRate IS Medium THEN AnimationSpeed IS Slow");
engine.NewRule("R_SlowButMedium_DmgMultiplier",
    "IF ReactionSpeed IS Slow AND SuccessRate IS Medium THEN DamageMultiplier IS Normal");

engine.NewRule("R_SlowButHigh_AnimSpeed",
    "IF ReactionSpeed IS Slow AND SuccessRate IS High THEN AnimationSpeed IS Normal");
engine.NewRule("R_SlowButHigh_DmgMultiplier",
    "IF ReactionSpeed IS Slow AND SuccessRate IS High THEN DamageMultiplier IS High");
}

public void EvaluatePlayer(float currentReaction, float currentSuccess)
{
    currentReaction = Mathf.Clamp(currentReaction, 0f, 2.0f);
    currentSuccess = Mathf.Clamp(currentSuccess, 0f, 100f);

    engine.SetInput("ReactionSpeed", currentReaction);
    engine.SetInput("SuccessRate", currentSuccess);

    float animSpeedMultiplier = engine.Evaluate("AnimationSpeed");
    float damageMultiplier = engine.Evaluate("DamageMultiplier");

    if (animSpeedMultiplier == 0) animSpeedMultiplier = 1.0f;
    if (damageMultiplier == 0) damageMultiplier = 1.0f;

    animSpeedMultiplier = Mathf.Clamp(animSpeedMultiplier, 0.7f, 1.2f);
    damageMultiplier = Mathf.Clamp(damageMultiplier, 0.5f, 1.5f);

    calculatedAnimSpeed = baseEnemySpeed * animSpeedMultiplier;
    calculatedDamageMod = baseEnemyDamage * damageMultiplier;

    if (_stateMachine != null)
    {
        _stateMachine.currentDdaSpeed = calculatedAnimSpeed;

        Animator anim = _stateMachine.GetComponent<Animator>();
        if (anim != null) anim.speed = calculatedAnimSpeed;

        if (_stateMachine.isPhase2)
        {
            calculatedDamageMod *= _stateMachine.phase2DamageMultiplier;
        }

        if (_stateMachine.axe != null)
        {
            _stateMachine.axe.damageAmount = (int) calculatedDamageMod;
        }

        if (_stateMachine.foot != null)
        {
            _stateMachine.foot.damageAmount = (int) (calculatedDamageMod / 2);
        }
    }

    Debug.Log($"Inputs -> Reaction {currentReaction}s | Success: {currentSuccess}%");
    Debug.Log($"Outputs -> Boss Anim Speed: {animSpeedMultiplier:F2}x | Boss Damage Multiplier: {damageMultiplier:F2}x");
}
}

```